

中国图书分类号 TP391;TP311

密级 公开

国际图书分类号 004

单位代码 10613

西南交通大学

硕士学位论文

面向政务领域的大模型数据查询与分析方 法研究与实现

学科专业 计算机科学与技术

学 号 2023201808

作者姓名 陈鑫海

指导教师 郑宇教授

学 院 计算机与人工智能学院

2026 年 3 月 20 日

A Thesis Submitted to
Southwest Jiaotong University for Master Degree

**Research and Implementation of Data Query and
Analysis Methods Based on Large Language Models
for the Government Affairs Domain**

Discipline Computer Science and
Technology

Student ID 2023201808

Author Chen Xinhai

Supervisor Professor Yu Zheng

School School of Computing and
Artificial Intelligence

March. 4, 2026

西南交通大学

学位论文独创性声明

本人郑重声明：所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得西南交通大学或其它教育机构的学位或证书而使用过的材料。其他人员对本研究所做的任何贡献均已在论文中作了明确的说明并表示谢意。

本学位论文的主要创新点如下：

作者签名：_____

日期： 年 月 日

西南交通大学

学位论文使用授权

本学位论文作者完全了解西南交通大学有关保留、使用学位论文的规定，同意学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅。本人授权西南交通大学可以将学位论文的全部或部分内容编入有关数据库进行检索及下载，可以采用影印、缩印、扫描等复制手段保存、汇编本学位论文。

本学位论文属于

1. 保密□，在 年解密后适用本授权书
2. 不保密□，使用本授权书。

(请在以上方框内打“√”)

作者签名：_____ 导师签名：_____

日期： 年 月 日

摘 要

这里是中文摘要输入区。

关键词：关键词 1，关键词 2，关键词 3，关键词 4，关键词 5

Abstract

Here is for you to write the English abstract.

Keywords: keyword1, keyword2, keyword3, keyword4, keyword5

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.1.1 研究背景	1
1.1.2 研究意义	1
1.2 国内外研究现状	1
1.2.1 基于大模型的 Text-to-SQL 研究现状	1
1.2.2 基于大模型的代码生成与自动数据分析研究现状	1
1.2.3 政务数据管理与应用平台研究现状	1
1.2.4 现有研究评述	1
1.3 论文研究内容	1
1.4 论文章节安排	1
1.5 本章小结	1
第 2 章 相关理论与技术	2
2.1 大语言模型基础理论	2
2.1.1 Transformer 模型结构与自注意力机制	2
2.1.2 预训练语言模型与指令微调	2
2.1.3 上下文学习与思维链推理	2
2.1.4 大语言模型的工具调用能力	2
2.2 文本生成 SQL 技术	2
2.2.1 Text-to-SQL 任务定义与技术演进	2
2.2.2 数据库模式表示与 Schema Linking	2
2.2.3 基于预训练模型的 Text-to-SQL 方法	2
2.2.4 基于大语言模型提示的 Text-to-SQL 方法	2
2.3 文本转数据分析技术	2
2.3.1 Text-to-Analysis 的基本概念与处理流程	2
2.3.2 面向数据分析的任务规划与分解	2
2.3.3 基于代码生成与工具调用的数据分析方法	2
2.3.4 数据分析结果的解释生成与可视化表达	2
2.4 多智能体协同与上下文管理技术	2
2.4.1 多智能体系统的基本概念与协作模式	2

2.4.2 面向复杂任务的 Agent 角色划分与任务编排	2
2.4.3 多轮交互中的上下文表示与记忆机制	2
2.5 本章小结	2
第 3 章 基于逻辑增强与异构协同的政务数据查询生成方法	3
3.1 引言	3
3.2 总体框架设计	4
3.3 离线知识准备	5
3.3.1 向量库构建	5
3.3.2 政务领域知识库构建	6
3.3.3 动态样例库构建	7
3.4 基于逻辑增强的模式链接	9
3.5 异构候选生成	12
3.5.1 逻辑映射生成器	12
3.5.2 渐进分治生成器	13
3.6 基于执行反馈的查询修复	14
3.7 候选判别模型	16
3.7.1 训练样本构建	17
3.7.2 判别模型训练	17
3.7.3 配对判别与最优候选选择	18
3.8 实验与结果分析	19
3.8.1 数据集构建与实验设置	19
3.8.2 评估指标与对比基线	20
3.8.3 总体性能对比	21
3.8.4 消融实验	22
3.8.5 异构协同与候选判别分析	23
3.8.6 效率分析	25
3.8.7 公共数据集实验结果	26
3.9 本章小结	28
第 4 章 基于分析规划与查询增强的政务数据分析方法	29
4.1 引言	29
4.2 总体框架设计	30
4.3 分析规划构建	32
4.3.1 规划模式定义	32

4.3.2 约束与完备性	32
4.3.3 规划生成	33
4.4 基于查询增强的分析任务分解方法	36
4.4.1 查询层与分析层的任务划分	36
4.4.2 任务分解准则与层级判定	37
4.4.3 查询结果表示与层间衔接	37
4.5 基于分析算子的结果生成方法	38
4.5.1 分析算子设计与编排	38
4.5.2 结果生成流程与输出组织	39
4.6 实验与结果分析	41
4.6.1 实验设置与评价指标	41
4.6.2 结果对比	43
4.6.3 分类型性能对比	43
4.6.4 模块消融实验	44
4.6.5 AP-Schema 规划质量评估	46
4.7 本章小结	46
第 5 章 系统设计与实现	48
5.1 引言	48
5.2 系统总体架构设计	48
5.3 业务流程设计	48
5.3.1 智能查询业务流程	48
5.3.2 智能分析业务流程	48
5.3.3 异常回退与结果追溯流程	48
5.4 系统数据存储	48
5.4.1 业务数据源接入与元数据抽取	48
5.4.2 系统数据库设计	48
5.4.3 结果、日志与缓存管理	48
5.5 多智能体协同与上下文管理实现	48
5.5.1 轻量级多智能体角色设计	48
5.5.2 查询与分析服务编排	48
5.5.3 上下文维护与状态流转	48
5.6 系统功能实现与效果展示	48
5.7 本章小结	48

结论与展望	49
致 谢	50
参考文献	51
附录 A	53
攻读硕士学位期间取得的研究成果	55

第 1 章 绪论

本章旨在阐述研究的宏观背景、理论与实践意义，并明确研究的主要内容、方法与整体结构。

1.1 研究背景与意义

1.1.1 研究背景

1.1.2 研究意义

1.2 国内外研究现状

1.2.1 基于大模型的 Text-to-SQL 研究现状

1.2.2 基于大模型的代码生成与自动数据分析研究现状

1.2.3 政务数据管理与应用平台研究现状

1.2.4 现有研究评述

1.3 论文研究内容

1.4 论文章节安排

1.5 本章小结

第 2 章 相关理论与技术

本章旨在详细介绍论文所依赖的核心理论与关键技术，为后续章节的方法设计和系统实现提供理论基础。

2.1 大语言模型基础理论

2.1.1 Transformer 模型结构与自注意力机制

2.1.2 预训练语言模型与指令微调

2.1.3 上下文学习与思维链推理

2.1.4 大语言模型的工具调用能力

2.2 文本生成 SQL 技术

2.2.1 Text-to-SQL 任务定义与技术演进

2.2.2 数据库模式表示与 Schema Linking

2.2.3 基于预训练模型的 Text-to-SQL 方法

2.2.4 基于大语言模型提示的 Text-to-SQL 方法

2.3 文本转数据分析技术

2.3.1 Text-to-Analysis 的基本概念与处理流程

2.3.2 面向数据分析的任务规划与分解

2.3.3 基于代码生成与工具调用的数据分析方法

2.3.4 数据分析结果的解释生成与可视化表达

2.4 多智能体协同与上下文管理技术

2.4.1 多智能体系统的基本概念与协作模式

2.4.2 面向复杂任务的 Agent 角色划分与任务编排

2.4.3 多轮交互中的上下文表示与记忆机制

2.5 本章小结

第3章 基于逻辑增强与异构协同的政务数据查询生成方法

3.1 引言

文本到 SQL 生成 (Text-to-SQL) 是将自然语言问题自动转换为可在关系数据库上执行的结构化查询语言的语义解析任务^[1], 是实现“智能问数”和降低数据分析门槛的关键技术。在政务领域, 随着数字政府建设的深入, 海量的人口、经济、社会等异构数据急剧增长, 使得非技术背景的公务人员能够直接、高效地从数据中获取决策支持显得尤为迫切。

本章提出的 Text-to-SQL 方法, 其任务形式化定义为: 给定用户的自然语言问题 Q 、目标数据库模式 S 、领域知识库 K 以及动态样例库 F , Text-to-SQL 任务的目标是生成一条可正确执行并返回预期结果的 SQL 查询语句, 即与用户意图最大化一致:

$$y^* \equiv \arg \max_{y \in Y} P(y \mid Q, S, K, F) \quad (3-1)$$

其中 Y 为符合 SQL 语法的候选集合, θ 为模型参数。

本章的研究目标是面向政务数据查询这一具体应用场景, 以系统性解决复杂政务语义下查询生成的正确性、执行成功率与鲁棒性问题。以一个典型的政务查询为例, 当用户提出“2023 年各区县的一般公共预算收入同比增长率是多少?”而这一看似简单的问题时, 系统需要理解“一般公共预算收入”对应的物理字段、“同比增长率”隐含的跨年度计算逻辑以及“区县”维度的分组聚合方式, 才能生成正确的 SQL 查询。这一过程涉及的业务知识远超数据库模式本身所能表达的信息边界。

与通用领域的 Text-to-SQL 任务相比, 政务数据查询面临着更为突出的复杂性挑战, 主要体现在以下方面:

(1) 业务术语的语义模糊与别名多样。政府工作存在大量专业术语及其简称、别名和口语化表达。例如, “一般公共预算收入”可能被简称为“一般预算收入”或“公共预算收入”, 而数据库中的物理字段名可能为 `ybsr` 等缩写形式。这种多对一的映射关系显著增加了模式链接的难度。

(2) 指标计算逻辑的隐含性。大量政务指标并非数据库中的直接存储字段, 而需要通过特定的计算公式推导得出。例如, “同比增长率”需要通过当期值与上期值的差值除以上期值来计算, “预算执行率”需要执行数除以预算数。这些隐含的

计算逻辑未显式存在于数据库模式中，导致大语言模型在面对此类查询时极易产生“逻辑幻觉”——即生成语法正确但业务逻辑错误的 SQL 语句。

(3) 查询结构的复杂性与多样性。SQL 查询的复杂度分布呈现明显的长尾特征：高频需求多为结构相对固定的简单查询，而低频需求则可能涉及多层嵌套子查询、跨表关联、条件聚合等复杂 SQL 结构。单一的生成范式难以同时兼顾这两类需求，对简单查询可能引入不必要的推理冗余，对复杂查询则可能因推理深度不足而导致准确率骤降。

上述问题分别对 Text-to-SQL 流程中的模式链接、候选生成和结果判别三个核心环节产生影响。基于此，本章的方法设计围绕以下三个目标展开：第一，通过构建领域知识库并将其显式注入数据库模式，同时设计一个应用于快速理解政务领域的数据库模式，以提升模式链接环节对政务语义的理解准确性；第二，通过设计异构协同的双路生成机制，平衡高频标准需求与低频复杂长尾需求的生成效率与质量；第三，通过引入执行反馈修复机制与基于微调的候选判别模型，提升最终查询结果的稳定性与鲁棒性。

3.2 总体框架设计

如图3-1所示，本章提出的基于逻辑增强与异构协同的 Text-to-SQL 方法包含离线知识准备和在线推理生成两条链路，由四个核心模块组成：离线数据预处理模块、模式链接模块、候选生成模块和候选判别模块。

离线阶段负责为在线推理提供高可用的辅助数据支撑。具体而言，该阶段从业务数据库中提取字符型列及其取值构建向量索引数据库 (*VecDB*)，为后续的值检索提供基础；通过人工梳理与结构化组织，构建包含业务术语映射、逻辑计算公式和值域约束的政务领域知识库 K ；同时，利用大语言模型对已有的“问题-SQL”对进行知识解构，生成包含中间推理链的动态样例库 F 。

在线阶段以用户的自然语言查询为输入，依次经过以下处理流程。首先，模式链接模块从原始数据库模式中检索与问题相关的表、列和值信息，并注入领域知识库中的业务术语、计算公式和值域约束，将原始数据库模式 S 转化为面向当前问题的逻辑增强模式 (LA-Schema) S^* 。随后，候选生成模块采用异构协同策略，通过逻辑映射生成器和渐进分治生成器两条并行推理路径，分别从领域逻辑映射和渐进式分解推理的角度生成多样化的 SQL 候选集。每个生成器基于大模型较高温度设置下并行采样生成候选 SQL 查询。对于生成过程中出现的语法错误或执行异常，查询修复器利用执行反馈信息引导大语言模型进行自反思修复。最终，候选判别模块通过微调的二分类选择模型对候选集进行配对比较与得分聚合，选出

最优的 SQL 查询语句作为最终输出。

各模块之间的数据流转关系形成了“知识准备—模式增强—候选生成—修复优化—判别选择”的完整闭环，其中离线知识准备为在线推理持续提供领域先验支撑，在线推理的执行结果又可反馈用于丰富样例库，实现系统能力的渐进增强。

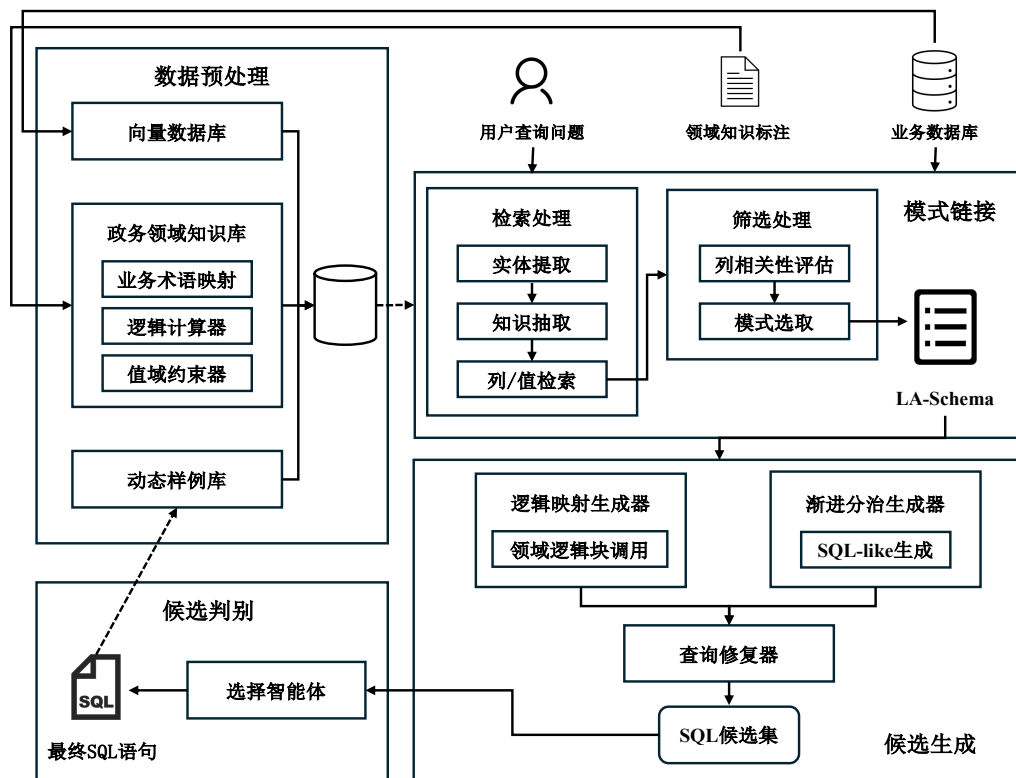


图 3-1 逻辑增强与异构协同的 Text-to-SQL 框架图

3.3 离线知识准备

离线数据预处理模块的目标是为后续在线推理生成提供低冗余、高可用的辅助数据。本模块通过构建多维度的数据知识支撑体系，为后续模式链接和候选生成过程提供精确的语义和逻辑基础。该支撑体系包含向量索引数据库、政务领域知识库以及动态样例库三个核心组件。

3.3.1 向量库构建

为保障大语言模型生成的 SQL 语句与底层数据库实际状态的一致性，本模块构建了高质量的向量索引数据库 *VecDB*，用于模式链接过程中的值检索。

具体构建流程如下。首先，从数据库中提取所有字符型列 C 及其对应的取值 v ，构造联合文本序列 $s = [C : v]$ 。通过将值的向量表示携带其所属列的上下文信

息，以帮助在检索时区分不同列下的同名值。

然后，利用预训练编码模型 BGE-m3^[2]将上述文本序列映射为高维语义向量 $\mathbf{h}$ 。此外考虑到政务数据库的规模庞大，为优化存储开销并提升检索效率，仅对字符串类型数据进行向量化。所有向量存入基于分层可导航小世界图（Hierarchical Navigable Small World, HNSW）^[3]的索引结构中。HNSW 通过构建多层跳跃链接的近邻图结构，在高维空间中实现对数级别的近似最近邻检索，相较于暴力搜索在大规模向量库上具有显著的效率优势，同时保持较高的召回率。最终向量库构建过程可表述为：

$$\begin{cases} \mathbf{h} = \text{BGEEncode}([C : v]) \\ \text{VecDB} = \{(\text{HNSW}(\mathbf{h}), C, v) \mid \forall C, v\} \end{cases} \quad (3-2)$$

向量数据库在在线阶段承担两方面的核心作用：一是通过语义相似度检索实现对用户问题中关键实体的精准召回，即使存在拼写差异或表述变化也能准确定位对应的数据库值；其次是为动态样例检索提供嵌入向量基础，使系统能够根据当前问题的语义特征检索最相关的历史样例。

3.3.2 政务领域知识库构建

针对政务数据查询中业务逻辑复杂且语义模糊的问题，本文构建了结构化的政务领域知识库 $K = \{B, L, V\}$ 。不同于传统的非结构化文本注释方式，该知识库采用结构化三元组形式存储先验业务知识，旨在为后续的模式链接和候选生成过程提供确切的逻辑增强。

知识库由以下三个组件构成：

(1) 业务术语映射 B ：记录政务领域中复杂业务指标与物理数据库字段之间的对应关系。例如，三元组（“一般公共预算收入”， $ybsr$ ，“一般公共预算收入金额字段”）将业务术语映射到具体的数据库字段。

$$B = \{(t_i, c_i, d_i)\}_{i=1}^{|B|} \quad (3-3)$$

其中 t_i 为业务术语， c_i 为对应的物理字段或字段组合， d_i 为转换说明。

(2) 逻辑计算器 L ：存储复杂的计算公式及聚合逻辑，将其表示为 SQL 代数表达式。例如，对于“同比增长率”这一指标，其 SQL 表达式为（当期值 - 上期值）/ 上期值 * 100。通过将复杂的业务计算逻辑预先编码为 SQL 代数表达式并以虚拟列的形式注入数据库模式 Schema，模型在生成 SQL 时可以直接调用这些预定义的逻辑块，从而规避了模型在处理大规模复杂计算公式时的推理冗余和盲

目判断。

$$L = \{(m_j, f_j, e_j)\}_{j=1}^{|L|} \quad (3-4)$$

其中 m_j 为指标名称, f_j 为计算公式的 SQL 表达式, e_j 为计算说明。

(3) 值域约束器 V : 记录特殊业务字段的取值范围或枚举值。例如, ($xzqh$, {“荣华街道”, “博兴街道”, “经济开发区”, ...}) 记录行政区划名称字段的合法取值。值域约束的引入能够帮助模型生成包含正确 WHERE 条件的 SQL 语句, 避免因值不匹配导致的查询结果为空。

$$V = \{(c_k, R_k)\}_{k=1}^{|V|} \quad (3-5)$$

其中 c_k 为字段名, R_k 为该字段的合法取值范围或枚举值集合。

知识库的来源包括政务业务术语表、字段文档、业务统计口径规则以及领域专家整理的历史问答知识。其标注过程以人机协同方式, 人工标注为主, 辅助以大语言模型辅助标注。知识的归一化组织遵循“术语—标准表达—对应字段/取值—业务说明”的统一结构, 便于在模式链接阶段进行高效检索与注入。

3.3.3 动态样例库构建

少样本 (Few-shot) 学习是辅助大语言模型进行 SQL 生成的一种重要方法。研究表明, 示例问题的质量和与当前问题的相关性对 Text-to-SQL 任务的生成效果具有关键影响^[4-5]。基于此, 本模块采用动态少样本学习策略, 而非固定的少样本集合, 以提高大语言模型在不同查询场景下的适应性和生成质量。

动态样例库的构建流程如图3-2所示。首先, 利用大语言模型对原始的“问题-SQL”对进行知识解构, 生成中间层的思维链 (Chain-of-Thought, CoT) 信息。该过程将每个样例从 $\langle \text{Query}, \text{SQL} \rangle$ 二元组扩展为包含完整推理逻辑的 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 三元组。以此构建一个逻辑表达能力更强的样例库。CoT 信息的具体组成如表3-1所示。这种结构化的推理链不仅为后续生成器提供了更丰富的推理线索, 也使得模型能够通过模仿样例中的推理过程来提升自身的逻辑表达能力。

表 3-1 思维链 (CoT) 信息的组成要素

要素	标识	说明
查询意图分析	reason	对用户问题的语义理解与意图解读
关键列	columns	查询涉及的数据库列
筛选值	values	WHERE 等筛选条件中使用的具体值
SELECT 内容	SELECT	SELECT 子句应返回的字段与表达式
类 SQL 语句	SQL-like	忽略连接条件的简化 SQL 骨架

而在在线推理阶段，当接收到用户查询 Q 时，系统启动端到端的动态样例检索流程，即从样例库 F 中选取与当前问题最相关的 k 个样例，以构建高质量的少样本提示。该流程融合了掩码问题相似性（Masked Question Similarity, MQS)^[4]与查询结构相似性的双重度量，从问法相似和 SQL 相似两个维度综合筛选样例。

具体而言，检索过程包含以下步骤。首先，对用户查询 Q 执行掩码操作，将其中的领域特定实体（如表名、列名、具体值）替换为统一占位符，得到掩码后的查询 Q' ：

$$Q' = \text{Mask}(Q) \quad (3-6)$$

随后，同样利用预训练编码模型 BGE-m3 将 Q' 映射为稠密向量表示 $v_{Q'} = \text{Enc}(Q')$ ，并在同样经过掩码处理的样例问题向量库中，使用基于 HNSW 的近似 k 近邻检索（Approximate Nearest Neighbor, ANN）算法召回一批候选样例集合 F_{cand} 。掩码操作使得相似度计算更关注问题的结构与语义特征，而非表面的实体匹配，从而有效提升跨数据库场景下的样例检索质量。

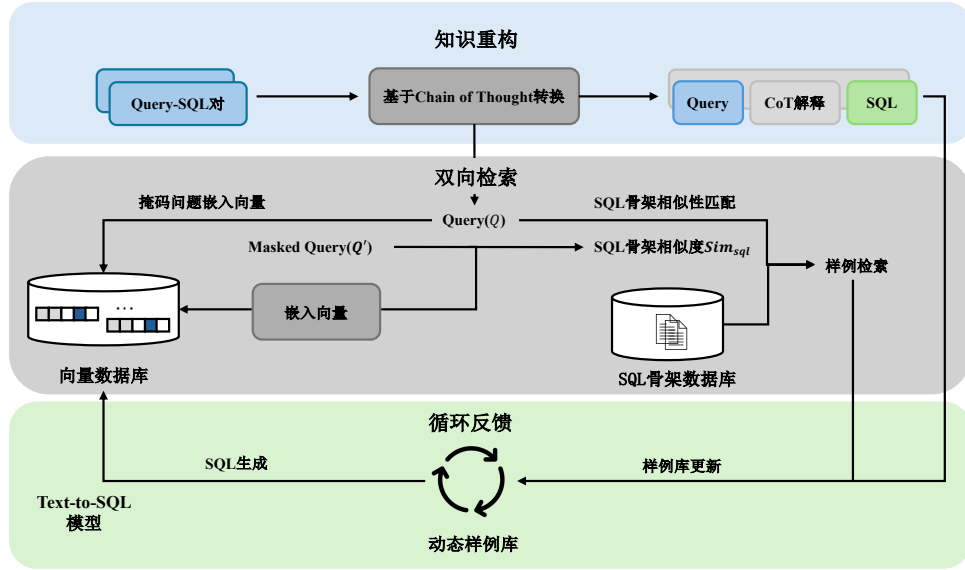


图 3-2 动态样例库构建流程图

其次在候选样例召回的基础上，系统进一步引入查询结构相似度对候选样例进行精排。具体地，先利用一个初步预测模型为 Q 生成近似 SQL 查询 $\hat{s}_Q$ 作为目标 SQL 的估计，然后将 $\hat{s}_Q$ 与每个候选样例的 SQL 查询 s_i 根据 SQL 关键词编码为二元离散语法向量，计算结构相似度 $\text{Sim}_{\text{sql}}(\hat{s}_Q, s_i)$ 。最终的样例排序综合考虑问题相似度与查询相似度，并通过预定义阈值 τ 过滤查询相似度不足的样例：

$$F_k = \text{Top-}k \left\{ \text{Sim}_{\text{ques}}(v_{Q'}, v_{Q'_i}) \mid \text{Sim}_{\text{sql}}(\hat{s}_Q, s_i) > \tau \right\}_{f_i \in F_{\text{cand}}} \quad (3-7)$$

其中 Sim_{ques} 为掩码问题向量间的相似度, F_k 即为最终选取的 k 个动态样例。选定的样例以 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 三元组的形式拼接至提示模板中, 与数据库模式、检索到的相似值及领域规则共同构成完整的少样本提示, 以引导大语言模型生成目标 SQL。

最终的动态样例库具备渐进更新的能力。当渐进分治生成器产生的候选 SQL 被判别模型确认为最优答案后, 该查询及其对应的推理过程可作为新的样例加入样例库 (可控制是否开启), 实现样例库的持续丰富与质量提升。

3.4 基于逻辑增强的模式链接

模式链接是 Text-to-SQL 任务中的关键环节, 其目标是将用户的自然语言查询与数据库模式中的元素 (包括表、列和值) 相关联, 从庞大的数据库模式中筛选出最小必要且包含丰富语义信息的数据库子模式^[6]。本节提出一种基于逻辑增强的模式链接方法, 通过动态检索元素信息、选取关键模式片段并注入领域逻辑信息, 将原始数据库模式 S 转化为面向特定问题 Q 的最小增强模式 S^* 。

该方法的处理流程包含检索、筛选和增强三个阶段, 其完整算法描述如算法3.1所示。

检索阶段旨在从数据库中搜索与查询潜在关联的值和列。该阶段首先以用户查询 Q 、动态样例 F_u 及领域知识库 K 为输入, 利用大语言模型识别用户问题中的关键词与实体, 获取实体集合 W_{entity} 。随后, 对 W_{entity} 中的每个实体 w 分别执行列检索与值检索。列检索基于实体 w 与列描述之间的语义相似度, 筛选每个实体对应的 Top- k 候选列。值检索基于 HNSW 索引从向量数据库中快速召回语义最近邻候选集 V_{cand} , 并利用嵌入向量的余弦相似度进行精排, 以实现目标值及其所属列的精准定位。检索到的候选列、匹配值及其所属表共同构成候选模式片段集合 S_{cand} 。

筛选阶段旨在将检索得到的表、列、值集合缩减为生成 SQL 所需的最小数据库模式。此过程根据检索得到的候选列元素集合, 让大语言模型评估每个列与用户查询的相关性, 过滤无关列, 最终得到最小数据库模式 S_{filter} 。该筛选步骤能够有效降低后续生成阶段的输入规模, 减少因冗余信息导致的模型注意力分散和幻觉问题。

增强阶段是本方法的核心创新所在。在获得最小数据库模式后, 系统从领域知识库 K 中提取与当前查询相关的业务知识, 并将其显式注入模式表示中, 形成逻辑增强模式 LA-Schema。LA-Schema 的组织方式参照 M-Schema^[7]的半结构化格式, 以层次化的方式展示数据库、表和列之间的结构关系, 并在此基础上添加政务

算法 3.1: 基于逻辑增强的模式链接算法

Input: 用户查询 Q , 原始数据库模式 S , 列检索召回数 k , 动态样例 $F_u = \{f_1, f_2, \dots, f_k\}$, 领域知识库 $K = \{B, L, V\}$, 向量数据库 VecDB, 大语言模型 LLM

Output: 最小增强模式 S^*

```

1 初始化: 候选模式片段集合  $S_{\text{cand}} \leftarrow \emptyset$ ;
   // 阶段一: 检索
2   $W_{\text{entity}} \leftarrow \text{LLM}(Q, F_u, K)$ ; // 实体提取
3  foreach  $w \in W_{\text{entity}}$  do
4       $C_{\text{topk}} \leftarrow \text{SemanticSearch}(w, C, k)$ ; // 列检索
5       $V_{\text{cand}} \leftarrow \text{HNSW}(w, \text{VecDB})$ ; // 值近邻召回
6       $\text{val} \leftarrow \arg \max_{v \in V_{\text{cand}}} \text{Sim}(\text{Enc}(w), \text{Enc}(v))$ ; // 语义精排
7       $S_{\text{cand}} \leftarrow S_{\text{cand}} \cup \{C_{\text{topk}}, \text{val}, \text{Table}(\text{val})\}$ ;
8  end
   // 阶段二: 筛选
9   $S_{\text{filter}} \leftarrow \text{LLM}(Q, S_{\text{cand}})$ ; // 列相关性评估与筛选
   // 阶段三: 增强
10  $C_{\text{alias}}, C_{\text{virtual}}, D_{\text{prompt}} \leftarrow \text{GetKnowledge}(W_{\text{entity}}, B, L, V)$ ;
11  $S^* \leftarrow \text{TransLASchema}(S_{\text{filter}}, C_{\text{alias}}, C_{\text{virtual}}, D_{\text{prompt}})$ ; // 注入知识并转换为
   LA-Schema
12 return  $S^*$ 

```

领域业务知识的显式表达。具体的增强内容包括三个层面:

(1) **术语归一与别名扩展:** 对于知识库 B 中与当前查询匹配的业务术语映射, 在对应字段上添加 **Business Term** 标记, 标注该字段在业务语义上的对应关系。例如, 当字段 `ybsr` 被识别为“一般公共预算收入”的物理存储字段时, LA-Schema 中会以 `[Business Term: 一般公共预算收入  $\rightarrow$  ybsr]` 的形式进行标注。

(2) **虚拟逻辑列注入:** 对于知识库 L 中匹配的计算指标, 在 LA-Schema 中创建虚拟列 (Virtual columns), 并附带预定义的 SQL 计算公式。例如, 对于“同比增长率”指标, 系统会注入一个标记为 `[Logical Term: 同比增长率, Formula: (当期值 - 上期值) / 上期值 * 100]` 的虚拟字段。这样, 大语言模型在生成 SQL 时可以直接引用预定义的计算逻辑, 而无需从零开始推理复杂的业务公式。

(3) **列值提示与约束:** 对于知识库 V 中匹配的值域约束以及通过值检索获得的匹配值, 在对应列上添加 **Value Term** 标记, 提示模型在 WHERE 条件中使用正确的匹配值。

为直观说明 LA-Schema 的增强效果, 以下给出一个对比示例。对于查询“帮我找出所有状态为已完工的民生项目, 并计算它们的资金结余率。”, 原始 DDL Schema、M-Schema 与 LA-Schema 的展示结果如表3-2所示。

表 3-2 原始 DDL Schema、M-Schema 与 LA-Schema 对比示例

模式类型	表示内容
DDL Schema	<pre>CREATE TABLE t_gov_proj(p_id INTEGER PRIMARY KEY, p_name VARCHAR, amt_tot INTEGER, amt_act INTEGER, p_type TINYINT, status TINYINT, p_year INTEGER)</pre>
M-Schema	<pre>[DB_ID] gover_database [Schema] # Table: t_gov_proj [(p_id: INTEGER, Primary Key, 项目编码, Examples:[1001,1002,1003]), (p_name: VARCHAR, 项目名称, Examples:[" 经开区智慧城市规划服务项目", "T 小区安居提升工程项目", "E 社区服务中心数字化基础设置建设"]), (amt_tot: INTEGER, 总概算, Examples:[1000000,2000000,3000000]), (amt_act: INTEGER, 实际支出, Examples:[100000,200000,300000]), (p_type: TINYINT, 项目类型, Examples:[1,2,3]), (status: TINYINT, 状态, Examples:[0,1]), (p_year: INTEGER, 年份, Examples:[2023,2024,2025])]</pre>
LA-Schema	<pre>[DB_ID] gover_database [Schema] # Table: t_gov_proj [Physical Columns: (p_id: INTEGER, Primary Key, 项目编码, Examples:[1001,1002,1003]), (p_name: VARCHAR, 项目名称, Examples:[" 经开区智慧城市规划服务项目", "T 小区安居提升工程项目", "E 社区服务中心数字化基础设置建设"]), (amt_tot: INTEGER, 总概算, Examples:[1000000,2000000,3000000]), (amt_act: INTEGER, 实际支出, Examples:[100000,200000,300000]), (p_type: TINYINT, 项目类型, Examples:[1,2,3]), Business Term:{" 民生项目": " 民生工程"}, Value Term:{" 民生工程":1, " 基础设施":2, " 产业扶持":3}), (status: TINYINT, 状态, Examples:[0,1]), Value Term:{" 进行中":0, " 已完工":1}), (p_year: INTEGER, 年份, Examples:[2023,2024,2025]) Virtual Columns: (balance_rate: Logical Term, 资金结余率, Examples:[0.1,0.2,0.3], Formula:"(amt_tot-amt_act)/amt_tot")]</pre>

可以看到，原始 DDL Schema 仅提供了最基本的列名与数据类型，缺乏任何语

义信息；M-Schema 在此基础上补充了中文列描述和示例值，但仍无法表达业务术语映射、值域约束等领域知识。而 LA-Schema 通过显式注入 Business Term、Value Term 与 Virtual Columns 等增强信息，使大语言模型能够准确理解“民生项目”对应的物理字段取值、“已完工”的编码映射以及“资金结余率”的计算公式，从而显著降低模式链接阶段的错误率。

3.5 异构候选生成

在政务数据查询场景中，用户查询的复杂度分布呈现显著的长尾特征^[8]，即高频需求往往是结构相对固定的标准查询，而低频需求可能涉及多层嵌套、多表关联和复杂聚合等高难度 SQL 结构。单一的生成路径在处理这种高度异构的查询分布时存在固有局限：对于简单查询，过度复杂的推理策略会引入冗余步骤并增加出错风险；对于复杂查询，直接映射式的生成方式又难以捕获深层的逻辑关系^[6]。

为此，本文设计了一种异构协同生成框架，包含旨在捕获高频业务逻辑的逻辑映射生成器以及处理复杂推理深度的渐进分治生成器。通过并行驱动两个推理范式截然不同的生成器，构建多样化的 SQL 候选池，以提升系统对各种复杂度查询的覆盖能力。生成阶段允许大语言模型在非零温度^[9]设置下执行随机采样，每个生成器并行生成 n_c 个候选 SQL 查询，最终总计产生 $2n_c$ 个多样性的 SQL 候选集。

3.5.1 逻辑映射生成器

逻辑映射生成器深度集成了本章提出的 LA-Schema 架构，其推理机制建立在领域逻辑映射的基础之上。该生成器通过引导大语言模型对 LA-Schema 中的领域先验进行识别与调用，其包括业务术语的语义对齐、虚拟逻辑列的公式引用以及值域约束的条件匹配。以有效地将复杂的 SQL 构造过程从底层的算子推导转化为对预设逻辑块的检索与组装。其完整流程如算法3.2所示。

算法中，BuildRules() 构建的映射规则指令明确要求模型遵循以下约束：（1）优先使用 LA-Schema 中标记为 Virtual Column 的虚拟逻辑列，直接在 SQL 中调用其预定义的计算公式；（2）识别并转换 Business Term 标记的业务术语，使用映射后的物理字段替换原始表达；（3）利用 Value Term 标记的匹配列值，确保 WHERE 条件中使用与数据库一致的标准取值；（4）直接输出最终的可执行 SQL，不作过多中间解释。生成阶段在非零温度下对同一提示执行 n_c 次独立采样，以获得多样化的候选 SQL 集合 $\mathcal{C}_{lm}$ 。

该生成器的核心优势在于效率高、对常见查询模式适应性强。对于政务场景

算法 3.2: 逻辑映射生成算法

Input: 用户查询 Q , LA-Schema 增强模式 S^* , 动态样例 $F_u = \{f_1, f_2, \dots, f_k\}$, 采样数 n_c , 大语言模型 LLM

Output: 候选 SQL 集合 $\mathcal{C}_{lm}$

```

1 初始化: 候选集合  $\mathcal{C}_{lm} \leftarrow \emptyset$ , 提示  $P \leftarrow \emptyset$ ;
   // 构建提示模板
2  $P \leftarrow P \oplus \text{FormatFewShot}(F_u)$ ;           // 拼接动态少样本示例
3  $P \leftarrow P \oplus S^*$ ;                         // 拼接 LA-Schema 增强模式
4  $R \leftarrow \text{BuildRules}()$ ;                     // 构建映射规则指令
5  $P \leftarrow P \oplus R \oplus Q$ ;                   // 拼接规则指令与用户查询
   // 多次采样生成候选 SQL
6 for  $i \leftarrow 1$  to  $n_c$  do
7    $\hat{y}_i \leftarrow \text{LLM}(P, \text{temperature} > 0)$ ; // 随机采样生成 SQL
8    $\mathcal{C}_{lm} \leftarrow \mathcal{C}_{lm} \cup \{\hat{y}_i\}$ ;
9 end
10 return  $\mathcal{C}_{lm}$ 

```

中高频出现的指标查询、条件筛选和简单聚合等标准需求, 逻辑映射生成器能够充分利用 LA-Schema 中预置的领域逻辑, 快速而准确地完成 SQL 生成, 避免了不必要的推理链条展开。

3.5.2 渐进分治生成器

与逻辑映射生成器的直接映射策略不同, 渐进分治生成器采用渐进式生成与动态少样本的联合方式, 以应对知识库未覆盖的复杂需求, 如多层嵌套逻辑、多条件组合聚合以及跨表关联查询等。

渐进式生成策略本质上是一种面向 Text-to-SQL 任务定制化的思维链方法^[10], 其核心在于将复杂的 SQL 生成任务拆解为多维度、结构化的子任务序列, 引导大语言模型通过逐步推理深度理解查询逻辑。该策略借鉴了 CHASE-SQL^[6]中的分治思想和 OpenSearch-SQL^[5]中的 SQL-Like 中间表示设计, 但针对政务场景做了两方面适配: 一是在推理链条中保留了 LA-Schema 的逻辑增强信息, 使分步推理能够参考领域知识; 二是引入了类 SQL 中间表示, 允许模型先构建查询骨架再填充语法细节, 降低了一次性生成完整 SQL 的复杂度。其完整流程如算法3.3所示。

算法中, 渐进推理指令 BuildCoTInstr() 要求模型在单次生成中按照表3-1所定义的结构化格式, 以自回归方式依次输出意图分析 (reason)、涉及列 (columns)、筛选值 (values)、SELECT 内容以及 SQL-like 骨架共五个部分, 前序输出自然成为后续生成的上下文, 形成递进式的推理链。随后, 系统将生成的 SQL-like 骨架作为上下文, 再次调用大语言模型补充完整的 JOIN 条件和语法细节, 生成最终可

算法 3.3: 渐进分治生成算法

Input: 用户查询 Q , LA-Schema 增强模式 S^* , 动态样例 $F_u = \{f_1, \dots, f_k\}$, 采样数 n_c , 大语言模型 LLM

Output: 候选 SQL 集合 $\mathcal{C}_{pd}$

- 1 初始化: 候选集合 $\mathcal{C}_{pd} \leftarrow \emptyset$, 提示 $P \leftarrow \emptyset$;
// 构建提示模板
- 2 $P \leftarrow P \oplus \text{FormatCoTShot}(F_u)$; // 拼接 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 格式示例
- 3 $P \leftarrow P \oplus S^*$; // 拼接 LA-Schema 增强模式
- 4 $P \leftarrow P \oplus \text{BuildCoTInstr}() \oplus Q$; // 拼接渐进推理指令与用户查询
// 多次采样生成候选 SQL
- 5 **for** $i \leftarrow 1$ **to** n_c **do**
 // 意图分析 $\rightarrow$ 涉及列与值 $\rightarrow \text{SELECT} \rightarrow \text{SQL-like}$
6 $(\text{reason}_i, \text{cols}_i, \text{vals}_i, \text{sel}_i, \hat{y}'_i) \leftarrow \text{LLM}(P, \text{temperature} > 0)$;
7 $\hat{y}_i \leftarrow \text{LLM}(P, \hat{y}'_i)$; // 基于 SQL-like 生成最终 SQL
8 $\mathcal{C}_{pd} \leftarrow \mathcal{C}_{pd} \cup \{\hat{y}_i\}$;
9 **end**
- 10 **return** $\mathcal{C}_{pd}$

执行的 SQL 查询。动态少样本示例以完整的 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 三元组呈现, 为模型提供渐进推理的范式参考。生成阶段同样在非零温度下执行 n_c 次独立采样, 产生多样化的候选集合 $\mathcal{C}_{pd}$ 。

该生成器的核心优势在于对复杂查询的处理能力。当查询涉及多层嵌套逻辑、复杂的条件组合或跨表关联时, 分步推理机制能够引导模型逐步构建查询结构, 避免因一步到位生成长 SQL 而导致的逻辑遗漏或结构错误。同时, 渐进生成器输出的结构化推理过程本身也具有可解释性, 有助于后续的错误分析和结果追溯。

此外, 渐进分治生成器与动态样例库之间存在正向反馈关系。当该生成器产生的候选 SQL 经判别后被确认为正确答案时, 其完整的 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 输出可作为新的高质量样例纳入动态样例库, 为后续相似问题的处理提供更精准的参考。

3.6 基于执行反馈的查询修复

无论是逻辑映射生成器还是渐进分治生成器, 其产生的 SQL 候选集在某些情况下不可避免地存在逻辑或语法错误^[6,11], 这些错误会导致查询无法正确执行或返回空结果。为提升候选集的整体质量, 本文引入了一个基于执行反馈的查询修复模块, 作为候选生成后、判别选择前的鲁棒性增强环节。

在实际政务查询场景中, 常见的 SQL 错误类型包括: (1) 字段名不存在或拼写错误, 通常由模式链接阶段的遗漏或大语言模型的幻觉引起; (2) 表连接关系错误, 如遗漏必要的 JOIN 条件或使用了错误的连接键; (3) SQL 语法错误, 如括

号不匹配、关键字使用不当等；（4）聚合逻辑错误，如 GROUP BY 子句缺失必要字段或聚合函数使用不当。

查询修复器的工作机制基于自反思（Self-Reflection）方法^[12]。整个过程如图3-3所示。对于候选集中的每条 SQL 查询，系统首先在目标数据库上执行该查询，捕获执行结果或错误信息。若执行成功则保留该候选；若执行失败（如语法错误），则将执行器的反馈信息包括具体的错误描述以及原始用户问题、LA-Schema 信息一并构造为修复提示，送入大语言模型进行反思与修复。修复后的 SQL 会再次执行以验证其有效性。

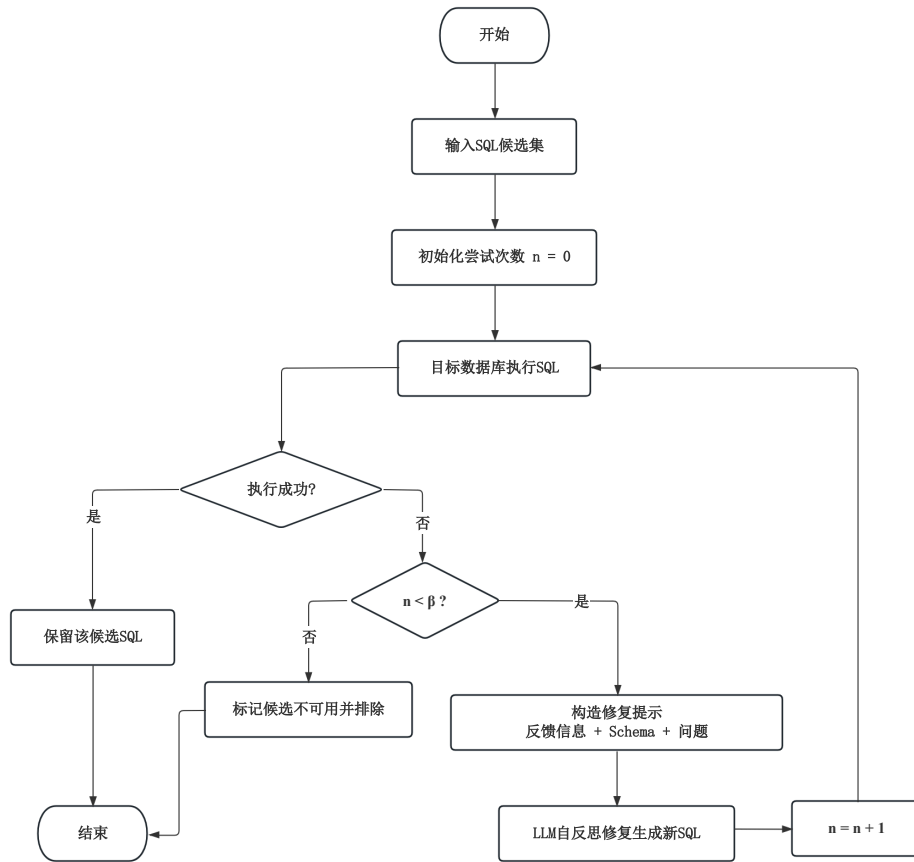


图 3-3 查询修复流程图

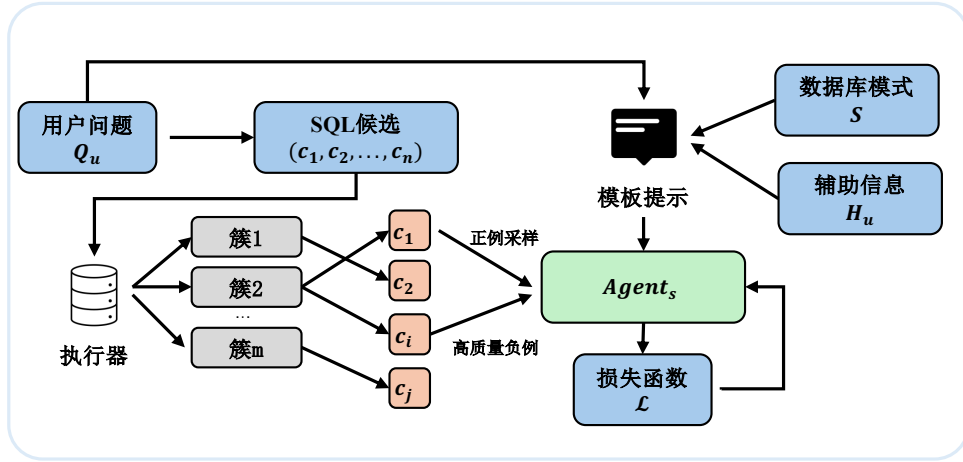
为避免无限迭代带来的效率损耗和不收敛风险，修复过程设置了最大尝试次数 β （本文设定 $\beta = 3$ ）。当满足以下任一条件时，修复过程终止：（1）修复后的 SQL 执行成功；（2）达到最大尝试次数 β 。若修复在 β 次尝试后仍未成功，则保留最后一次修复的结果或标记该候选为不可用，在后续判别阶段予以排除。

查询修复器的引入有效降低了候选集中的无效查询比例，为后续的候选判别模块提供了更高质量的输入，从而间接提升了最终查询结果的准确率。

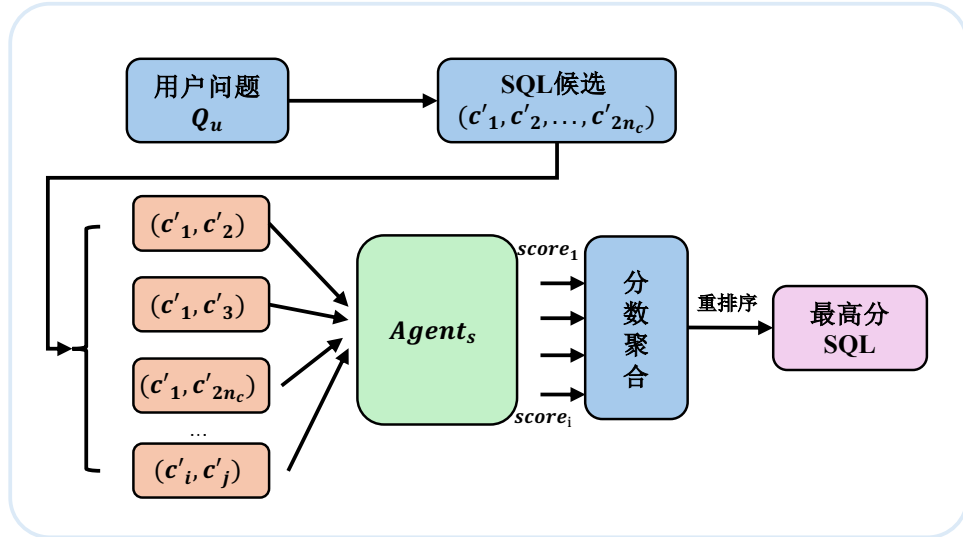
3.7 候选判别模型

经过异构候选生成与查询修复后，系统可为每个用户问题构建包含 $2n_c$ 个候选 SQL 的查询池。本节的目标是从该候选集中可靠地选出最终正确的 SQL 查询。

现有许多 Text-to-SQL 方法采用候选查询的一致性投票（Self-Consistency）策略，以得票最多的结果作为最终输出^[5]。然而，在复杂业务查询场景中，最一致的答案并不总是正确答案^[6]，其一致性选择策略与上界精度存在一定的性能差距。为此，本文构建选择智能体 $Agent_s$ ，并将候选选择建模为配对偏好判别问题。具体而言， $Agent_s$ 以用户问题、辅助提示信息、相关增强模式以及一对候选 SQL 为输入，判断两者中哪一个更可能正确，再通过两两比较与得分聚合确定最终输出。整体流程如图3-4所示。



(a) 离线训练



(b) 在线推理

图 3-4 候选判别整体流程

3.7.1 训练样本构建

判别模型的训练数据来源于在训练集上运行候选生成模块所得到的候选 SQL 集合。对于训练集中的每个问题 Q_u ，系统首先利用两个异构生成器产生 $2n_c$ 个候选 SQL，并在目标数据库上执行所有候选查询。设标准 SQL 的执行结果为 R_u^* ，候选 c_k 的执行结果为 R_k 。系统依据执行结果对候选进行聚类，将执行结果相同的候选划分至同一簇，再将各簇对应的执行结果与 R_u^* 进行比较：若二者一致，则该簇记为正确簇；否则记为错误簇。

在同时存在正确簇与错误簇的情况下，系统从两类簇中抽取候选，构造配对训练样本：

$$T_{ij} = (Q_u, H_u, c_i, c_j, S_{ij}^*, y_{ij}), \quad (3-8)$$

其中， Q_u 表示用户问题， H_u 表示辅助提示信息， c_i 与 c_j 表示两个待比较的候选 SQL， S_{ij}^* 表示 c_i 和 c_j 所涉及增强数据库模式的并集， $y_{ij} \in \{0, 1\}$ 为监督标签。当 c_i 来自正确簇且 c_j 来自错误簇时，记 $y_{ij} = 1$ ；反之记 $y_{ij} = 0$ 。

在样本构建过程中重点挖掘高价值负例。所谓高价值负例，是指执行结果与正确答案接近、但查询逻辑存在关键偏差的错误 SQL，例如筛选条件边界不同、聚合范围不一致或连接关系错误等。相较于随机负例，这类样本更能反映真实业务场景中的判别难点。对于训练集中未能生成正确候选的问题，本文仅在离线样本构造阶段引入标准 SQL 的执行结果，用于引导生成器产生更多与正确答案相似但不完全相同的错误候选，从而提升高价值负例的比例；该信息不作为判别模型在线推理时的输入，以避免信息泄漏。

3.7.2 判别模型训练

系统以 Qwen-8B 为基础模型构建选择智能体 Agent_s ，并采用指令式监督微调方式^[13](Instruction Supervised Fine-Tuning, ISFT) 进行训练。对于每个训练样本 T_{ij} ，系统将用户问题 Q_u 、辅助提示信息 H_u 、模式并集 S_{ij}^* 以及候选 SQL(c_i, c_j) 按统一模板组织为输入序列，要求模型输出二元判别结果，以表示两者中哪一个更可能为正确查询。由此， Agent_s 学习的是如下条件偏好概率：

$$p_\theta(y_{ij} = 1 \mid Q_u, H_u, S_{ij}^*, c_i, c_j), \quad (3-9)$$

其中 θ 表示大模型经微调后的模型参数。

训练目标采用二分类交叉熵损失函数：

$$\mathcal{L}_{\text{pair}} = -y_{ij} \log p_\theta(y_{ij} = 1 \mid T_{ij}) - (1 - y_{ij}) \log p_\theta(y_{ij} = 0 \mid T_{ij}). \quad (3-10)$$

考虑到高价值负例对模型判别能力的提升更为显著, 本文进一步引入样本权重 w_{ij} , 得到总体优化目标:

$$\mathcal{L} = \sum_{(i,j)} w_{ij} \mathcal{L}_{\text{pair}}. \quad (3-11)$$

其中, 对语义接近但逻辑错误的高价值负例赋予更高权重, 以增强模型对细微差异的敏感性。为兼顾训练效率与模型性能, 本文采用参数高效微调策略对大模型进行训练, 使其学习候选 SQL 之间的相对优劣关系且不丢失基础模型理解能力。

3.7.3 配对判别与最优候选选择

在线推理阶段, 系统基于配对比较与得分聚合机制, 从候选集 $C = \{c_1, c_2, \dots, c_{2n_c}\}$ 中选出最终 SQL, 其完整流程如算法3.4所示。

算法 3.4: SQL 候选集选取算法

Input: 候选集 $C = \{c_1, c_2, \dots, c_{2n_c}\}$, 用户问题 Q_u , 目标数据库 DB , 增强数据库模式 S^* , 选择智能体 Agent_s

Output: 最佳 SQL 查询 c_f

```

1 foreach  $c_i \in C$  do
2   |  $\text{score}_i \leftarrow 0$ ;
3 end
4 foreach  $(c_i, c_j) \in C \times C$  且  $i \neq j$  do
5   | if  $\text{ans}(c_i, DB) = \text{ans}(c_j, DB)$  then
6   |   |  $\text{winner} \leftarrow i$ ; // 结果一致时, 无需调用判别模型
7   | else
8   |   |  $S_{ij}^* \leftarrow \text{SchemaUnion}(c_i, c_j, S^*)$ ;
9   |   |  $\text{winner} \leftarrow \text{Agent}_s(Q_u, S_{ij}^*, c_i, c_j)$ ;
10  | end
11  |  $\text{score}_{\text{winner}} \leftarrow \text{score}_{\text{winner}} + 1$ ;
12 end
13  $c_f \leftarrow \arg \max_{c_i \in C} \text{score}_i$ ;
14 return  $c_f$ 

```

对于任意两个不同候选 (c_i, c_j) , 系统首先在目标数据库 DB 上执行二者并比较执行结果。若 $\text{ans}(c_i, DB) = \text{ans}(c_j, DB)$, 则说明二者在功能上等价, 无需调用判别模型; 若二者执行结果不同, 则构造其相关增强模式并集 $S_{ij}^* = \text{SchemaUnion}(c_i, c_j, S^*)$, 并调用 Agent_s 进行二元判别。考虑到输入顺序可能影响模型输出, 系统对每一对

候选同时比较 (c_i, c_j) 与 (c_j, c_i) 两个方向，并将胜者分别累加得分，从而减轻顺序偏差。最终，系统选择累计得分最高的候选作为输出：

$$c_f = \arg \max_{c_i \in C} \text{score}_i. \quad (3-12)$$

若出现平局，系统优先选择执行成功且返回非空结果的候选；若仍无法区分，则进一步选择 SQL 语句长度较短的候选作为最终结果。通过上述机制，本文将候选选择由一致性投票转化为基于配对偏好的全局竞争过程，从而更有效地利用候选间的细粒度差异信息，提高最终 SQL 的选取准确率。

3.8 实验与结果分析

本章首先介绍实验设置，包含数据集构建与选用、评估指标定义、对比模型。然后针对政务领域数据集上进行对比实验、消融实验、异构协同与判别分析及效率分析。最后本文还在公共数据集 BIRD^[14]、Spider^[15]上进行实验，以评估本文方法在泛用数据集上的适用性与稳健型。

3.8.1 数据集构建与实验设置

为全面评估本文方法在政务领域的有效性，实验采用自建政务数据集进行评估。鉴于政务数据的隐私敏感性，本文基于“多源采集—逻辑注入—混合标注”的流程构建了面向政务场景的 Text-to-SQL 评测数据集。

数据集的构建流程分为三个阶段：

(1) **多源数据采集与脱敏**。数据来源于某市经济开发区政务平台，原始数据涵盖财政预算、公共资源和政务项目三个领域的 5 个数据库，共计 33 张数据表。为保障数据安全，对涉及个人隐私的敏感信息（如身份证号、联系方式等）进行了加盐哈希的严格脱敏处理，确保数据在技术上不可逆向还原。

(2) **基于领域知识库的逻辑注入**。通过人工梳理 20 余个核心业务指标，将包括预算执行率、同比增长率、集中采购率等在内的隐性业务逻辑显性化地录入标注元数据中。确保数据集包含足够多的需要领域知识。

(3) **人机协同混合标注**。问答对的生成采用人机协同的方式：首先利用 Qwen 大语言模型^[16]根据原始数据库模式生成种子问题和初始 SQL，然后由具备政务业务背景的标注人员对生成结果进行核验和纠正，确保问题的自然性和 SQL 的正确性。

最终构建的数据集包含 480 条高质量的自然语言问题—SQL 查询对，按照 7:3 的比例划分为训练集和测试集。训练集用于候选判别模型微调，而在训练样本构

建阶段产生共计近 5000 个元组。测试集用于最终端到端性能评估。数据集中约 38.5% 的问题涉及复杂查询（包括多表连接、嵌套子查询、复杂聚合等）。数据集统计信息如表3-3所示。

表 3-3 政务数据集 GovSQL 统计信息

划分	领域数	数据库数	数据表数	字段数	问题-SQL 对数	复杂查询占比/%
训练集	3	5	33	217	336	37.8
测试集	3	5	33	217	144	40.3
合计	3	5	33	217	480	38.5

在实验配置方面, 本文使用 Qwen3-Coder-30B-A3B-Instruct^[16]作为基础调用大语言模型, 用于模式链接、候选生成和查询修复等主要环节; 使用 Qwen3-8B^[16]作为候选判别模型的基础模型进行微调训练。

存储引擎使用 Milvus 作为向量库构建存储, PostgreSQL 作为领域知识库, 并联合使用 Milvus 和 PostgreSQL 实现动态样例库。在工程上使用 Redis 作为热点缓存工具, 提高调用性能。联合使用 Milvus、PostgreSQL 作为向量库、知识库以及样例库的存储引擎。

实现设置上, 动态样例检索的样本数 n_f 设为 5, 此外样例库实验评估阶段关闭在线增量更新, 避免测试数据泄漏。模式链接阶段, 候选列的召回数 k 设为 10。除候选生成外的其他 LLM 调用阶段温度参数 $temperature = 0.3$ 。而在候选生成阶段, 每个生成器在温度参数 $temperature = 0.7$ 的设置下各采样生成 $n_c = 10$ 个候选 SQL。

3.8.2 评估指标与对比基线

本文采用执行准确率 (Execution Accuracy, EX) 作为主要评估指标。EX 通过比较预测 SQL 查询与参考 SQL 查询在目标数据库上的执行结果来判定正确性:

$$EX = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{ans}(SQL_i^{\text{pred}}, DB) = \text{ans}(SQL_i^{\text{gold}}, DB)] \quad (3-13)$$

其中 N 为测试集样本数, $\text{ans}(SQL, DB)$ 表示 SQL 在数据库 DB 上的执行结果。EX 指标关注的是查询的功能等价性而非形式一致性, 即允许 SQL 在写法上存在差异, 只要执行结果相同即视为正确。

此外, 本文在异构协同与候选判别分析和效率分析中还使用了以下辅助指标: 判别模型的二分类准确率以及端到端平均响应时延。

对比基线分为以下三类：

(1) **开源基础大模型**。该类基线采用零样本（zero-shot）Text-to-SQL 提示直接生成 SQL，不引入任何额外的优化策略，旨在衡量大语言模型本身的 SQL 生成能力下限。具体包括：DeepSeek-V3^[17]，本地部署，采用零样本提示生成 SQL；Qwen3-Coder-30B-A3B-Instruct^[16]，本地部署，同样采用零样本提示生成 SQL。

(2) **基于提示词的无微调方法**。该方法通过精心设计的提示工程策略提升 SQL 生成质量，无需对模型参数进行微调。具体包括：DAIL-SQL^[4]，通过为不同问题检索语义相似的问题-SQL 对作为少样本示例，引导大语言模型生成 SQL；OpenSearch-SQL^[5]，通过动态上下文学习与一致性对齐策略，以多智能体协同的方式生成 SQL；Alpha-SQL^[18]，基于蒙特卡洛树搜索建模 Text-to-SQL 生成过程，配合动态生成推理动作与自监督奖励函数进行搜索式 SQL 生成。

(3) **基于监督微调的方法**。该方法通过在 Text-to-SQL 数据上对模型进行监督微调，使模型习得面向 SQL 生成的专用能力。具体包括：XiYan-SQL^[7]，融合监督微调与上下文学习，并使用 M-Schema 增强模式理解，以多生成器集成的方式生成 SQL；CHASE-SQL^[6]，通过多路径候选生成器产生多样化的 SQL 候选，并使用经偏好优化微调的选择代理进行候选判别；OmniSQL^[19]，通过自动化合成百万级高质量 Text-to-SQL 数据及思维链标注，基于开源大模型进行大规模监督微调。

为保证对比的公平性，上述基线方法（除开源基础大模型和 OmniSQL 外）均使用与本文相同的 Qwen3-Coder-30B-A3B-Instruct 作为基础 LLM。

3.8.3 总体性能对比

表3-4展示了各方法在政务数据集上的执行准确率对比结果。本文方法取得了 91.47% 的执行准确率，显著优于所有对比基线。

表 3-4 不同 Text-to-SQL 方法在政务数据集上的性能对比

类别	方法	执行准确率/%
开源基础大模型	DeepSeek-V3	64.90
	Qwen3-Coder-30B-A3B-Instruct	69.17
基于提示词无微调	DAIL-SQL	74.42
	OpenSearch-SQL	76.51
	Alpha-SQL	80.27
基于监督微调	OmniSQL	71.56
	CHASE-SQL	81.33
	XiYan-SQL	<u>83.83</u>
本文方法	Ours	91.47

从结果中可以得出以下分析：

以 DeepSeek-V3 和 Qwen3-Coder 为代表的零样本方法,其准确率普遍处于 65% 至 70% 之间。分析发现,这类方法虽然具备一定的 SQL 语法生成能力,但在面对政务场景中晦涩的物理列名和复杂的业务指标时,极易产生“语义幻觉”,即模型臆造了不存在的列名或使用了错误的计算逻辑,无法准确映射底层的业务规则。

DAIL-SQL 和 Alpha-SQL 等基于提示词工程的方法通过优化上下文样本选择在一定程度上提升了性能,其中 Alpha-SQL 达到了 80.27%。然而,这类方法本质上仍属于隐式推理范式,模型必须在有限的上下文窗口内同时完成语义理解和逻辑推演。在处理高嵌套、跨表计算等复杂政务查询时,上下文中的有效信息容易被稀释,推理链条断裂的概率显著增加。

XiYan-SQL 和 CHASE-SQL 等包含微调组件的方法表现相对优异,分别达到了 83.83% 和 81.33%。这表明监督微调在特定领域数据集上确实能带来增益效果。但即便经过微调,如果模型不具备实时的领域逻辑增强支撑,在面对政务业务中涉及复杂计算公式和动态变化的业务规则时,仍然难以突破 85% 的准确率瓶颈。

相比之下,本文方法相较于最强基线 XiYan-SQL 实现了 7.64 个百分点的提升,相较于 CHASE-SQL 实现了 10.14 个百分点的提升。这一显著改善主要归因于三方面的协同作用: LA-Schema 的逻辑增强机制有效缓解了政务领域的语义幻觉问题;异构协同生成策略扩大了候选池的多样性和覆盖度;微调判别模型则从多样化的候选中精准筛选出最优答案。

3.8.4 消融实验

为深入探究系统中各核心模块的有效性,本文设计了消融实验。通过在完整模型 (Full Model) 的基础上分别移除或替换关键模块,观察其在政务数据集上的执行准确率变化。实验结果如表3-5所示,移除任何一个模块均会导致性能下降,证明了各模块的不可或缺性。

w/o HNSW。在数据预处理阶段,若不使用 HNSW 索引进行向量值检索,而直接采用语义匹配的单阶段值检索,执行准确率降至 89.32%。这证明了 HNSW 索引在处理大规模政务元数据时,能够更高效且精准地召回与查询相关的数据库值。

w/o Dynamic Few-shot。移除动态样例库后,模型仅依赖固定的提示词模板,准确率下降 6.16% 至 85.31%。这表明政务查询具有高度的领域多样性,通过动态检索语义相似的少样本样例,能够为模型提供关键的上下文生成参考,增强系统对特定业务语境的自适应能力。尤其是 $\langle \text{Query}, \text{CoT}, \text{SQL} \rangle$ 格式的样例比传统的 $\langle \text{Query}, \text{SQL} \rangle$ 格式能够提供更丰富的推理线索。

表 3-5 移除系统关键模块的消融实验性能比较

模型方法	执行准确率/%	ΔEX /%
Full Model	91.47	-
w/o HNSW	89.32	-2.15
w/o Dynamic Few-shot	85.31	-6.16
w/o LA-Schema	84.62	-6.85
w/o Logical Generator	87.93	-3.54
w/o DC Generator	88.88	-2.59
w/o Query Fixer	89.24	-2.23
w/o Selection Agent	86.12	-5.35

w/o LA-Schema. 当系统不使用逻辑增强模式而是采用传统的 DDL Schema 时, 性能出现最大的下滑, 降幅达 6.85%。值得注意的是, 该配置下的准确率 (84.62%) 与最强基线 XiYan-SQL (83.83%) 接近, 说明在去除 LA-Schema 后系统的性能优势几乎消失。这充分证明了在政务垂直领域, 物理字段名的隐晦性和业务逻辑的复杂性是 SQL 生成的核心瓶颈, 而 LA-Schema 通过显式注入业务等先验信息, 将复杂的逻辑演算简化为模式调用, 是系统高性能的核心保障。

w/o Logical Generator & w/o DC Generator. 分别去除逻辑映射生成器和渐进分治生成器后, 执行准确率分别下降 3.54% 和 2.59%。逻辑映射生成器的贡献略大于渐进分治生成器, 这反映了政务数据集中包含较多需要领域逻辑映射的业务指标查询。

w/o Query Fixer. 去除查询修复器后, 准确率下降 2.23% 至 89.24%。虽然修复器的单独贡献相对较小, 但其通过基于执行反馈的迭代修正机制, 有效提升了候选集中可执行查询的比例, 为后续判别模型提供了更高质量的输入。

w/o Selection Agent. 若不使用基于执行反馈的选择智能体进行候选判别, 而是采用一致性多数投票策略, 准确率下降 5.35% 至 86.12%。这说明多数投票无法充分感知 SQL 的执行质量, 而选择智能体能够通过执行反馈识别出语义虽不一致但逻辑正确的潜在答案, 显著提升了异构候选冲突时的容错率。

3.8.5 异构协同与候选判别分析

为深入剖析异构协同生成机制与候选判别模型这一核心创新的有效性, 从生成器独立性能、判别模型精度、候选池多样性三个维度设计了系统性的专项实验。

生成器性能分析

为揭示各生成器的独立贡献, 本文在候选判别前对每个生成器的单候选生成能力进行了评估。即将每个生成器生成单条候选 SQL 的准确率与零样本 CoT 基线

进行对比，同时考察查询修复器对各生成器的增益效果。结果如图3-5所示。

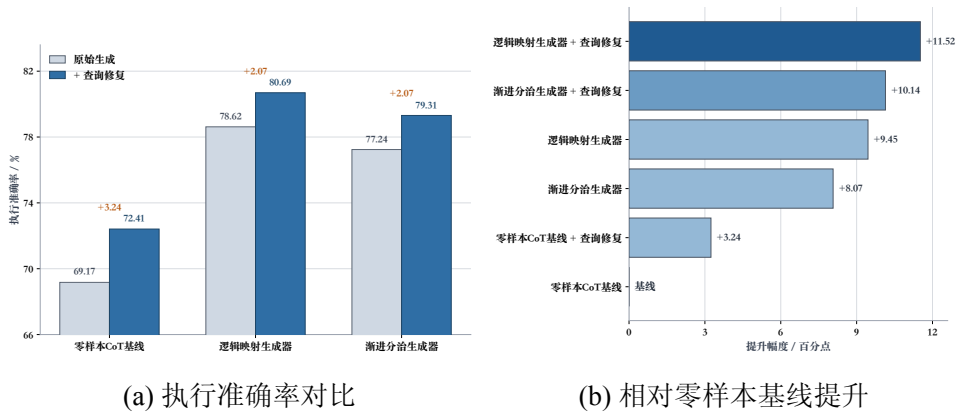


图 3-5 各生成器单候选生成性能对比

从结果中可以得出以下分析。首先，两个生成器的单候选性能均显著优于零样本 CoT 基线，逻辑映射生成器提升了 9.45 个百分点，渐进分治生成器提升了 8.07 个百分点，验证了本文所设计的生成策略在引导大语言模型进行 SQL 生成方面的有效性。其中逻辑映射生成器略优于渐进分治生成器，这主要归因于政务数据集中包含大量可直接利用 LA-Schema 预置逻辑块的业务指标查询。其次，查询修复器对所有生成方法均带来了约 2%~3% 的性能增益，表明基于执行反馈的迭代修复机制能够有效纠正语法错误和执行异常，提升候选的整体可用性。

判别模型分析

候选判别模型的二分类准确率直接决定了从候选池中选出正确答案的能力。本文对不同选择策略的判别精度进行了对比实验，在配对比较中仅统计一个候选正确、另一个候选错误的高价值情况。结果如表3-6所示。

表 3-6 不同选择策略的二分类判别准确率

选择模型	二分类准确率/%
Qwen3-Coder-30B（未微调）	59.38
Qwen3-8B（未微调）	55.62
Qwen3-8B（微调，本文方法）	72.85

实验结果表明，未经微调的大语言模型在配对判别任务上的准确率仅略高于随机猜测水平（55%~59%），这是因为异构生成器产生的候选 SQL 之间往往仅存在细微差异（如 WHERE 条件中的一个值不同、聚合范围略有差别等），未微调模型难以捕获这些关键区分特征。相比之下，经过在训练集上产生的高价值正负样本对进行微调后，Qwen3-8B 的判别准确率提升至 72.85%，提升了 17.23 个百分

点。这一结果与 CHASE-SQL^[6]中的发现一致，即配对选择任务需要通过监督微调使模型学习到候选 SQL 之间细粒度的语义偏好。

候选池多样性与生成器互补性分析

本文对两个生成器在候选池层面的互补性进行了深入考察。对于测试集中的每个问题，分别统计仅逻辑映射生成器产生正确候选、仅渐进分治生成器产生正确候选以及两者均产生正确候选的情况。如图3-6所示。

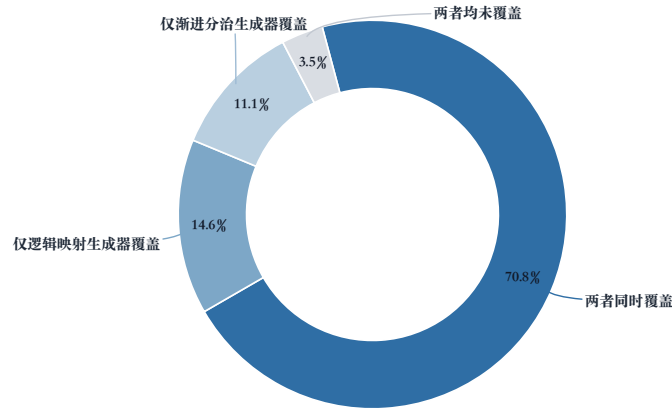


图 3-6 生成器在测试集覆盖统计

在 144 个测试样本中，有 21 个问题（14.6%）仅被逻辑映射生成器覆盖，16 个问题（11.1%）仅被渐进分治生成器覆盖，102 个问题（70.8%）被两个生成器同时覆盖，仅有 5 个问题（3.5%）未被任一生成器产生的候选所覆盖。

逻辑映射生成器独占的 21 个正确案例以包含预定义业务指标的标准化查询为主，涉及需要精确公式调用的问题。渐进分治生成器独占的 16 个正确案例则集中在结构复杂的查询上，包括多层嵌套子查询、跨表关联聚合等场景。两种推理范式在查询类型上的互补性是异构协同策略有效的根本原因。

3.8.6 效率分析

表3-7展示了本文方法各模块的平均耗时、时间占比、LLM 调用次数及 Token 消耗情况。

端到端单条查询存在波动，累计需 7~24 次 LLM 调用，Token 消耗范围为 12500~43000 个，具体取决于候选修复的轮次以及生成器生成的多样性。

其中候选生成环节占据了最大的时间比重（42.6%），平均耗时 17.1 秒，这主要是由于两个生成器各需采样多个候选并涉及较长的提示序列，尽管双路并行执

表 3-7 各模块效率分析（平均单样本）

模块	平均耗时/s	占比/%	平均 LLM 调用/次	Token 消耗/个
模式链接	7.8	19.5	3	3000–8000
候选生成（双路并行）	17.1	42.6	3	9000–20000
查询修复	3.8	9.5	0–3	0–6000
候选判别	10.7	26.7	1–15	500–9000
其他（SQL 执行等）	0.7	1.7	0	0
端到端总计	40.1	100.0	7–24	12500–43000

行已有效压缩了该环节的绝对耗时。每次候选生成固定调用 3 次 LLM（逻辑映射生成器 1 次、渐进分治生成器 2 次-均为线程数为 10 的并发调用），Token 消耗为 9000~20000 个，是所有模块中最高的。候选判别环节耗时 10.7 秒（26.7%），是第二大耗时模块。该环节需要对候选对执行多轮配对比较，LLM 调用次数为 1~15 次，波动较大的原因在于执行结果一致的候选对可直接跳过模型判别，而执行结果不同的候选对则需逐对调用选择智能体，Token 消耗为 500~9000 个。模式链接环节耗时 7.8 秒（19.5%），需 3 次 LLM 调用，其开销主要来自实体提取、向量检索及大语言模型的列相关性评估，Token 消耗为 3000~8000 个。查询修复环节耗时 3.8 秒（9.5%），LLM 调用次数为 0~3 次，Token 消耗为 0~6000 个。当所有候选 SQL 均执行成功时无需触发修复流程，调用次数和 Token 消耗均为零；当存在执行失败的候选时，最多进行 3 轮迭代修复。

对于政务数据查询这一非实时交互的应用场景而言，40 秒左右的响应时间处于可接受范围内。在实际部署中，候选生成的并行化策略（两个生成器同时运行且多线程生成候选 SQL）已将该环节的耗时控制在合理水平；同时 SQL 执行结果的缓存大大提高了使用性能。若需进一步降低延迟，可考虑减少每个生成器的采样数量（如降低至 5 或者 3），以在准确率与响应速度之间取得更优的折中。

3.8.7 公共数据集实验结果

BIRD 和 Spider 是两个广受认可的跨域数据集，分有不重叠的训练集、开发集、测试集。BIRD 包含来自 95 个大型数据库的超过 12,751 个独特的问题-SQL 对，涵盖 37 个专业领域，其数据库设计模仿现实世界场景，具有杂乱的数据行和复杂的模式。Spider 包含 10,181 个问题和 5,693 个独特的复杂 SQL 查询，涉及 200 个数据库，覆盖 138 个领域。

为评估本文方法在泛用场景下的适用性与稳健性，本文在两个主流的跨领域公共 Text-to-SQL 基准数据集上进行了实验：BIRD^[14]开发集（BIRD-dev）和 Spider^[15]测试集（Spider-test）。在公共数据集实验中，本文方法未使用政务领域知识

库，动态样例库也替换为基于目标数据集训练集构建的通用样例库，以确保评测条件与其他基线方法保持一致。各方法的执行准确率对比结果如表3-8所示。

表 3-8 公共数据集上的性能对比

方法	使用模型	BIRD/%	Spider/%
DeepSeek-V3	DeepSeek-V3	63.80	85.80
Qwen3-Coder	Qwen3-Coder-30B-A3B-Instruct	67.33	88.27
DAIL-SQL	GPT-4	54.76	86.60
OpenSearch-SQL	GPT-4o	69.30	87.10
Alpha-SQL	Qwen2.5-Coder-32B	69.70	-
XiYan-SQL	GPT-4o + Qwen2.5-Coder-32B	73.34	89.65
CHASE-SQL	Gemini-1.5-Pro + Gemini-1.5-Flash	74.90	87.60
OmniSQL	Qwen2.5-Coder-32B	67.00	89.80
本文方法	Qwen3-Coder-30B-A3B-Instruct + Qwen3-Flash-8B	71.23	89.25

从结果中可以得出以下分析：

(1) 在 **BIRD-dev** 上，本文方法取得了 71.23% 的执行准确率，仅次于 CHASE-SQL (74.90%) 和 XiYan-SQL (73.34%)。与政务数据集上的显著领先不同，本文方法在 BIRD 上未能超越这两个强基线，其核心原因在于：BIRD 数据集包含 95 个不同领域的数据库，本文的 LA-Schema 机制在缺乏目标领域知识库支撑的情况下无法发挥其领域逻辑增强的核心优势。而 CHASE-SQL 和 XiYan-SQL 针对通用场景设计了更具普适性的生成策略（如 CHASE-SQL 的查询执行计划 CoT 和 XiYan-SQL 的多生成器集成），在无领域先验的条件下表现更为稳健。尽管如此，本文方法仍显著优于 DAIL-SQL (+16.47%)、OmniSQL (+4.23%) 等方法，且相较于使用相同基础模型的 Qwen3-Coder 零样本基线提升了 3.90 个百分点，表明异构协同生成与微调判别机制本身在通用场景下同样具备增益能力。

(2) 在 **Spider-test** 上，本文方法取得了 89.25% 的执行准确率，仅次于 OmniSQL (89.80%) 和 XiYan-SQL (89.65%)，超过了 CHASE-SQL (87.60%)。Spider 数据集的查询结构相对规范、领域语义歧义较少，各方法的性能差距较 BIRD 上更为收窄。本文方法在未针对 Spider 做任何特定适配的情况下展现竞争力表现，说明本文设计的异构候选生成框架和配对判别机制具有较好的跨数据集泛化能力。值得注意的是，OmniSQL 通过百万级合成数据的大规模监督微调获得了最高性能，而本文方法仅对判别模型进行了小规模微调，在效率上更具优势。

(3) 综合来看，本文方法在公共数据集上表现出稳定的竞争力，验证了异构协同生成与微调判别这一技术框架的通用性。同时，本文方法在政务数据集上的领先优势 (91.47%) 与公共数据集上的性能差距也从侧面印证了 LA-Schema 领域逻

辑增强机制的核心价值，即在具备领域知识支撑的垂直场景中，本文方法能够充分发挥知识驱动的优势，实现显著超越；而在缺乏领域知识的通用场景中，方法仍能依靠异构生成与判别机制保持主流水平。

3.9 本章小结

本章针对政务数据查询中业务语义模糊、计算逻辑隐含及查询结构复杂等挑战，提出了基于逻辑增强与异构协同的 Text-to-SQL 方法。该方法通过构建结构化领域知识库并以 LA-Schema 形式显式注入数据库模式，提升政务领域的业务理解能力；通过逻辑映射与渐进分治双路径异构生成机制，平衡了高频标准需求与低频复杂长尾需求；通过微调二分类判别模型与执行反馈修复机制，提升了最终查询的准确性与鲁棒性。最终在政务数据集和公开数据集验证了本章所提模型的有效性。为构建智能问数系统提供了核心技术支持。

第 4 章 基于分析规划与查询增强的政务数据分析方法

4.1 引言

第三章围绕政务智能问数中的可靠取数问题，提出了基于逻辑增强与异构协同的 Text-to-SQL 方法，重点解决自然语言查询到结构化查询语句的准确映射问题。然而，在真实政务业务场景中，用户需求往往并不止于“查到数据”，而是进一步希望系统能够围绕查询结果完成趋势识别、横向比较、结构刻画和结论归纳等分析任务。因此仅有可靠的 SQL 生成能力仍不足以支撑完整的政务数据洞察。

基于此，本章聚焦于：如何在第三章可靠查询能力的基础上，将自然语言分析需求转化为结构化分析计划；如何将分析任务拆解为查询层与分析层两个相互协同但职责清晰的阶段；以及如何基于查询结果生成规范化的图表、表格和文字洞察。换言之，第三章解决查准，本章进一步解决围绕查询结果开展规范分析。

本章将分析过程定义为：给定自然语言分析需求 x 、目标数据库模式 S 、已构建的领域知识库 K 、动态样例库 F 、查询生成工作流 G_q 、分析算子库 O_a 以及可视化规则集合 $\mathcal{V}$ ，系统最终生成分析规划 A 和结果包 R ，使得分析结果与用户意图最大化一致，即：

$$(A^*, R^*) = \arg \max_{A \in \mathcal{A}, R \in \mathcal{R}} P(A, R \mid x, S, K, F, G_q, O_a, \mathcal{V}) \quad (4-1)$$

其中， $\mathcal{A}$ 表示候选分析规划集合， $\mathcal{R}$ 表示候选结果集合。为支撑上层应用展示与后续系统实现，本章将输出统一组织为

$$\text{ResultPackage} = \{\text{charts}, \text{tables}, \text{insights}, \text{logs}\} \quad (4-2)$$

其中 `charts` 表示图表文件或图表描述集合，`tables` 表示结构化分析表格结果，`insights` 表示文字洞察摘要，`logs` 表示执行状态与异常信息。

结合政务场景中的高频需求，本章将目标任务限定为四类典型分析类型，各类型的核心逻辑与适用场景如表4-1所示。

与查询任务相比，面向分析的结果生成面临三类挑战。其一，分析需求通常隐含指标、维度、时间粒度、统计口径和展示形式等多重约束，若缺乏中间规划过程，系统容易遗漏关键分析步骤；其二，若分析环节重新承担数据库理解与 SQL 生成任务，则会再次暴露 Schema 歧义、字段映射错误和逻辑幻觉等问题；其三，分析结果不仅要正确，还要展示规范，因此需要在方法层面形成统一的结果组织方式，而具体的服务部署等工程实现将在系统实现章节展开。

表 4-1 四类典型分析类型及其适用场景

分析类型	核心逻辑	适用场景
趋势分析	沿时间维度追踪指标的变化轨迹与演进规律	财政收入增减趋势、企业注册量波动、工单受理量走势等
对比分析	沿区域、部门、年份等维度对关键指标进行横向比较	不同区县经济指标差异、部门绩效对比、年度预算同比等
排名分析	按指定指标排序并识别头部与尾部对象	项目投资额排名、预算执行率排名、办件效率排名等
结构分析	计算整体内部各组成部分的占比与分布	财政支出结构、产业类型分布、收入来源构成等

基于上述考虑，本章以“分析规划、查询增强、算子生成、规范输出”为总体思路展开研究。首先，通过构建结构化中间表示 AP-Schema，将开放式分析需求转化为面向执行的分析规划；第二，通过查询增强的任务分解机制，将分析任务拆分为查询层操作与分析层操作，确保复杂业务口径仍由数据查询模块统一处理；第三，引入轻量分析算子库与可视化规则，在查询结果上完成稳定的分析结果生成；最后，将输出统一组织为标准化结果包，为后续的系统实现提供清晰的方法基础。通过上述设计，数据查询模块与数据分析模块共同构成从可靠取数到规范分析的完整技术链路。

4.2 总体框架设计

基于上述目标，本文设计了一个面向政务洞察的分析规划与查询增强协同框架。其整体流程由需求解析、AP-Schema 构建、查询需求投影、取数调用、分析算子执行以及结果组织六个阶段组成。与第三章的 Text-to-SQL 流程相比，本章不再重新研究 SQL 生成，而是将第三章的方法作为标准化数据获取子模块嵌入分析流程中，实现“先查准、再分析”的层级协同。

框架的核心数据流可表示为：

$$\begin{cases} A = \text{BuildPlan}(x, K), \\ Q_A = \pi_q(A), P_A = \pi_a(A), \\ D = G_q(x, Q_A; S, K, F), \\ R = \text{Generate}(P_A, D; \mathcal{O}_a, \mathcal{V}) \end{cases} \quad (4-3)$$

其中， A 表示由分析规划模块生成的 AP-Schema； $\pi_q(\cdot)$ 表示从 AP-Schema 中投影出取数需求子结构的操作； $\pi_a(\cdot)$ 表示投影出分析层操作与展示要求的操作； S 为

目标数据库模式； K 为3.3.2中构建的领域知识； F 为3.3.3构建的动态样例库； G_q 表示对第三章查询生成管线与数据库执行过程的上层抽象。具体而言，系统首先通过模板化组装函数将原始分析需求 x 与查询投影 Q_A 合成为受约束的查询子问题 $x_q = \text{Assemble}(x, Q_A)$ ，再结合目标数据库模式 S 、领域知识库 K 和动态样例库 F ，由数据查询方法（3.4至3.7）完成模式链接、候选生成与最优 SQL 查询 c_f 选择，最后在目标数据库上执行 c_f 并封装返回结果； $\mathcal{D}$ 表示该管线返回的数据结果、字段元数据与执行状态； R 表示最终结果包。

整个框架如图4-1所示：

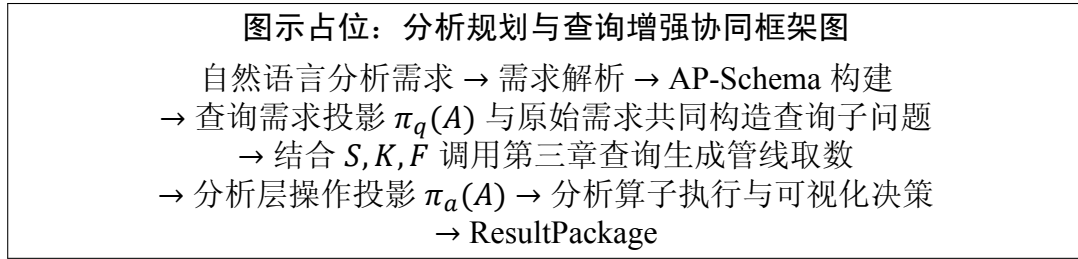


图 4-1 分析规划与查询增强协同数据分析框架图

需求解析阶段负责从用户输入中识别分析对象、指标、时间范围、分组维度和期望展示形式，为后续规划提供语义基础。

AP-Schema 构建阶段将自然语言需求转化为结构化分析表示，用于明确“分析什么”“通过哪些数据分析”“如何展示结果”三个核心问题。该中间表示是本章的关键方法载体，也是连接第三章查询方法与第五章系统实现的重要桥梁。

查询需求投影阶段从 AP-Schema 中抽取与数据获取相关的指标、维度、过滤条件、时间粒度和查询层操作，并调用查询生成管线完成可靠取数。

取数调用阶段依据 $x_q = \text{Assemble}(x, Q_A)$ 将受约束查询子问题送入第三章查询生成管线，以确保字段绑定、逻辑指标求解和 SQL 执行仍由查询模块统一完成。

分析算子执行阶段在获取到查询结果后，根据分析类型与任务要求，从轻量分析算子库中选择合适的操作链，对结果数据完成排序、占比、透视、标注等分析处理，并结合可视化规则生成规范化图表与表格。

结果组织阶段将图表、表格、摘要和日志统一封装为 ResultPackage，以便后续系统展示、日志记录和误差分析。

通过上述设计，本章的方法在保持大语言模型语义理解能力的同时，引入了清晰的中间表示、稳定的任务分解机制和规范化的结果生成流程，从而应对政务场景下对可靠性、规范性与可落地性的综合要求。

4.3 分析规划构建

分析规划并非对用户需求的简单转写，而是将分析对象、统计口径、执行边界与展示要求统一编码为可验证的中间表示。为此，本节围绕 AP-Schema (Analysis Plan Schema) 展开，依次说明其模式定义、字段约束与完备性校验机制，以及从自然语言需求到结构化规划实例的构建过程，从而为后续查询投影与分析算子编排提供统一基础。AP-Schema 的整个构建与约束机制过程如图4-2所示。

图 4-2 AP-Schema 构建与约束机制图

4.3.1 规划模式定义

为统一表达分析需求并为后续查询投影与算子编排提供结构化基础，本文引入分析规划模式 AP-Schema，并将其定义为：

$$A = \langle T, M, D, \hat{F}, G, O_q, O_a, \hat{V}, \hat{S} \rangle, \quad T \in \{\text{trend, comparison, ranking, structural}\} \quad (4-4)$$

表4-2给出了 AP-Schema 各字段的语义定义。与原始自然语言需求相比，AP-Schema 将分析任务中的关键约束显式化，使后续的查询投影、分析算子组织和结果验证能够围绕同一结构化表示协同展开。

表 4-2 AP-Schema 字段定义

组成部分	符号	含义	典型取值
分析类型	T	任务所属的分析类别	trend、ranking 等
指标集合	M	待分析指标及其聚合方式	预算收入/SUM、项目数/COUNT
维度集合	D	用于分组、对比或结构刻画的语义维度	区县、部门、时间、支出领域
筛选条件	$\hat{F}$	时间范围与对象过滤条件	年份=2025、区县 $\in \{A, B\}$
分析粒度	G	时间或统计粒度	年度、季度、月度、类别级
查询层操作	O_q	取数阶段的口径与关联操作	同比口径、跨表关联
分析层操作	O_a	结果上的展示与整理操作	sort→topk→annotate
可视化规格	$\hat{V}$	图表类型与坐标映射要求	折线图、柱状图、饼图
摘要要求	$\hat{S}$	文字洞察需覆盖的要点	峰值月份、占比前三、最大增幅

4.3.2 约束与完备性

AP-Schema 的九个字段并非相互独立，而是存在由分析类型 T 驱动的结构性约束。不同的 T 取值对维度集合 D 、分析粒度 G 和默认可视化类型 $\hat{V}$ 等字段提出

了不同的最低要求，从而将后续生成过程约束在合理的决策空间内。表4-3列出了四类分析类型对关键字段的典型约束。

表 4-3 分析类型对 AP-Schema 关键字段的约束

T 分析类型	D 必须包含	G 典型取值	$\hat{V}$ 默认	O_a 常用算子
趋势分析	时间维度	月度/季度/年度	折线图	sort, annotate
对比分析	至少一个对比维度	按对比维度聚合	柱状图	pivot, sort
排名分析	排序维度	按维度聚合	横向柱状图	sort, topk
结构分析	分类维度	类别级汇总	饼图/堆叠柱	share, annotate

这些约束在规划生成阶段起到完备性校验的作用。例如，若系统识别出 $T = \text{trend}$ 却未在 D 中包含时间维度，则规划应视为不完备并触发补全；若识别出 $T = \text{structural}$ 却未发现任何分类维度字段，系统需要回溯至意图识别阶段重新提取维度信息。本文将该校验形式化为：

$$\text{Valid}(A) = \bigwedge_{c_i \in \mathcal{C}} c_i(A) \quad (4-5)$$

其中， $\mathcal{C}$ 表示由表4-3推导的约束规则集合。当 $\text{Valid}(A)$ 不成立时，系统将尝试从用户输入中二次抽取缺失信息或使用领域知识推断默认值。

4.3.3 规划生成

AP-Schema 的构建过程本质上是从开放式自然语言分析需求到结构化规划表示的语义映射。该映射并非要求大语言模型自由输出分析结论，而是先通过固定槽位提示模板生成规划草案 $A^{(0)}$ ，再依次经过意图识别、槽位填充、操作分解与规格初始化四个子步骤，对草案中的字段进行解析、校验和补全，从而逐层完成对分析需求的结构化。

意图识别阶段负责确定分析类型 T 。系统通过大语言模型对用户输入执行语义解析，识别其中显式或隐式包含的分析意图线索，包括趋势词（“变化”“走势”“波动”）、比较词（“对比”“差距”“哪个更高”）、排序词（“排名”“前三”“最多”）和结构词（“占比”“构成”“分布”）。当输入中同时包含多类意图线索时，系统采用优先级规则进行消歧：若同时出现时间变化语义和对比语义，则优先判定为趋势分析（ $T = \text{trend}$ ），因为对比通常是趋势分析的附属展示而非独立任务。

槽位填充阶段负责从用户输入中提取 M 、 D 、 $\hat{F}$ 、 G 四个核心字段。该阶段结合领域知识库 $K = \{B, L, V\}$ 进行语义归一。具体而言，指标词通过业务术语映射 B 归一为标准业务指标及其默认聚合方式；时间范围和对象范围通过值域约束 V

确认取值合法性；分析粒度 G 根据时间词和分析类型联合推断。例如，当用户提出“分析近两年各区一般公共预算收入的月度趋势”时，“一般公共预算收入”先通过 B 归一为标准业务指标“一般公共预算收入/SUM”，而不在本阶段直接绑定到具体物理字段；“近两年”通过 V 映射为年份 $\in \{2024, 2025\}$ ，“月度”确定 $G = \text{month}$ 。后续在查询需求投影阶段，系统再将这些约束连同原始需求交由数据查询模块结合数据库模式 S 完成字段绑定与 SQL 生成。

操作分解阶段负责将用户需求中涉及的操作划分为查询层操作 O_q 与分析层操作 O_a 。其判断依据是：凡依赖数据库模式理解或改变统计口径的操作归入 O_q ，凡仅对已获取结果进行重组或展示的操作归入 O_a 。例如，上述需求中“并给出同比变化”涉及逻辑计算器 L 中预定义的跨年度计算公式，应映射为查询层逻辑指标 ($O_q = \{\text{同月同比口径}\}$)，而非在分析层进行自由计算。

规格初始化阶段根据已确定的 T 、 D 、 G 以及可视化规则库 $\mathcal{V}$ 生成初始可视化规格 $\hat{V}$ 和摘要要求 $\hat{S}$ 。此阶段利用表4-3中的默认映射关系确定初始图表类型，并根据指标数量和维度数量调整坐标轴分配与标题内容。

上述映射过程可统一表述为：

$$\begin{cases} A^{(0)} = \text{LLM}_{\text{plan}}(x, K, C), \\ A = \text{BuildPlan}(x, K) \end{cases} \quad (4-6)$$

AP-Schema 的完整生成流程如算法4.1所示。

固定槽位模板要求模型按照 $T \rightarrow M \rightarrow D \rightarrow \hat{F} \rightarrow G \rightarrow O_q/O_a \rightarrow \hat{V}/\hat{S}$ 的顺序输出草案，从而避免大语言模型在规划阶段直接生成冗长叙述文本。上述流程在返回前需进行基于公式(4-5)的完备性校验。当校验发现规划中存在字段缺失或类型不匹配时，系统回溯至槽位填充阶段进行二次提取，并对时间粒度、默认图表类型和摘要要点等字段执行规则化补全，从而提升规划的鲁棒性。

通过上述四步，系统可将“分析近两年各区一般公共预算收入的月度趋势，并给出同比变化”这一自然语言需求映射为表4-4所示的完整 AP-Schema 实例。

最终 AP-Schema 的引入带来三方面收益。其一，它将分析任务中的隐式约束显式化，减少系统在后续步骤中遗漏关键分析要求的风险；其二，它为数据查询与数据分析模块之间建立了统一接口，使“取什么数据”和“如何组织分析结果”能够在同一规划框架下协同工作；其三，字段约束关系和完备性校验为规划质量提供了可检查的保障机制，使系统在面对表述不完整或存在歧义的用户输入时仍能生成结构完整的分析规划。

算法 4.1: AP-Schema 构建算法

Input: 自然语言分析需求 x , 任务类型集合 $\mathcal{T}$, 领域知识 $K = \{B, L, V\}$, 分析算子库 $\mathcal{O}_a$, 可视化规则库 $\mathcal{V}$, 约束规则集 $\mathcal{C}$, 大语言模型 LLM

Output: 分析计划 $A = \langle T, M, D, \hat{F}, G, O_q, O_a, \hat{V}, \hat{S} \rangle$

```

1 初始化:  $A \leftarrow \emptyset$ ;
2  $A^{(0)} \leftarrow \text{LLM}(\text{PromptTemplate}(x, K, \mathcal{C}))$ ; // 按固定槽位输出规划草案
3  $E \leftarrow \text{ParseDraft}(A^{(0)})$ ; // 解析草案中的指标、维度、条件、粒度与展示意图
4  $T \leftarrow \text{IdentifyType}(E, \mathcal{T})$ ; // 识别分析类型
5  $(M, D, \hat{F}, G) \leftarrow \text{ParseSlots}(E, K)$ ; // 归一化指标、维度、条件与粒度
6  $(O_q, O_a) \leftarrow \text{DecomposeOps}(T, M, D, \hat{F}, G, K, \mathcal{O}_a)$ ; // 区分查询层与分析层操作
7  $\hat{V} \leftarrow \text{InitVis}(T, D, G, \mathcal{V})$ ; // 生成初始可视化规格
8  $\hat{S} \leftarrow \text{BuildSummarySpec}(T, M, \hat{V})$ ; // 生成摘要要求
9  $A \leftarrow \langle T, M, D, \hat{F}, G, O_q, O_a, \hat{V}, \hat{S} \rangle$ ;
10 if  $\neg \text{Valid}(A, \mathcal{C})$  then
11      $(M, D, \hat{F}, G) \leftarrow \text{CompleteFields}(x, K, T)$ ; // 基于约束规则触发补全
        // 再次执行重新获取字段信息
12      $(O_q, O_a) \leftarrow \text{DecomposeOps}(T, M, D, \hat{F}, G, K, \mathcal{O}_a)$ 
13      $\hat{V} \leftarrow \text{InitVis}(T, D, G, \mathcal{V})$ 
14      $\hat{S} \leftarrow \text{BuildSummarySpec}(T, M, \hat{V})$ 
15      $A \leftarrow \langle T, M, D, \hat{F}, G, O_q, O_a, \hat{V}, \hat{S} \rangle$ ;
16 end
17 return  $A$ 

```

表 4-4 自然语言分析需求到 AP-Schema 的映射示例

字段	映射结果
T	trend
M	{一般公共预算收入/SUM, 同比增长率/逻辑指标}
D	{区县, 时间}
$\hat{F}$	{年份 $\in \{2024, 2025\}$ }
G	月度 (month)
O_q	{按区县和月度汇总, 同月同比口径}
O_a	[sort, annotate]
$\hat{V}$	折线图, x 轴 = 时间, y 轴 = 收入金额, 系列 = 区县, 辅助指标 = 同比增长率
$\hat{S}$	趋势摘要: 整体走势、峰值月份、同比最大增幅

4.4 基于查询增强的分析任务分解方法

上一节完成了从自然语言需求到 AP-Schema 的结构化规划构建。本节进一步讨论如何将规划中的操作分配给查询层与分析层，并给出两层之间的结果衔接方式。具体而言，本节依次说明查询需求投影与分析需求投影的定义（4.4.1节）、操作归属的判定准则与执行流程（4.4.2节），以及查询层输出的标准结果表示（4.4.3节）。

4.4.1 查询层与分析层的任务划分

据^[8]研究发现以及实践表明，若查询与分析耦合职能划分不明确，则会出现逻辑幻觉重复计算等问题。故本章采用如下划分原则：涉及统计口径、复杂关联或底层数据库计算逻辑的操作，交由第三章查询生成管线完成；本章分析层执行作用于查询结果组织、展示与解释的操作。查询生成在给定用户问题 Q 、数据库模式 S 、领域知识库 K 和动态样例库 F 的条件下完成模式链接、候选生成与候选判别；本章则不直接展开 SQL，而是将与取数相关的约束显式化后交由查询层求解。

为此，本文从 AP-Schema 中分别定义查询需求投影函数与分析需求投影函数：

$$\begin{cases} Q_A = \pi_q(A) = \langle M, D, \hat{F}, G, O_q \rangle \\ P_A = \pi_a(A) = \langle O_a, \hat{V}, \hat{S} \rangle \end{cases} \quad (4-7)$$

其中， Q_A 表示分析规划中与取数相关的结构化约束，描述待查询的指标与聚合要求（ M ）、分组维度（ D ）、过滤条件（ $\hat{F}$ ）、时间粒度（ G ）以及需在查询阶段完成的业务口径操作（ O_q ）； P_A 表示分析层操作链（ O_a ）、可视化规格（ $\hat{V}$ ）和摘要要求（ $\hat{S}$ ）。

任务划分的执行流程可概括为四步，如图4-3所示。首先，从 AP-Schema 中抽取指标、维度、条件、粒度和候选操作，形成待判定的取数约束与分析约束；其次，围绕模式依赖性、统计口径依赖性和查询结构复杂性依次判断各操作的归属；再次，将满足查询层条件的部分汇总为 Q_A ，将仅依赖结果组织的部分汇总为 P_A ；最后，由查询层输出标准结果表示 $\mathcal{D}$ ，分析层再据此执行算子调用、可视化映射和摘要生成。由此，任务划分从原则性描述转化为可执行的过程表示。

图示占位：查询层与分析层任务划分流程图

自然语言分析需求 $\rightarrow$ AP-Schema
 $\rightarrow$ 查询需求投影 Q_A 与分析需求投影 P_A
 $\rightarrow Q_A + \text{原始需求} + S, K, F \rightarrow \text{第三章查询生成管线} \rightarrow \text{查询结果表示 } \mathcal{D}$
 $\rightarrow P_A + \mathcal{D} \rightarrow \text{分析算子与可视化规则} \rightarrow \text{ResultPackage}$

图 4-3 查询层与分析层任务划分执行流程图

4.4.2 任务分解准则与层级判定

为了在具体操作层面系统地区分 O_q 与 O_a ，本文给出如下判定准则：

$$\phi(o) = \begin{cases} \text{query-layer,} & \text{若 } o \text{ 改变统计口径或依赖底层模式与关系计算} \\ \text{analysis-layer,} & \text{若 } o \text{ 仅对已获取结果集进行重组、展示或解释} \end{cases} \quad (4-8)$$

该准则可进一步落实为四个判定维度：操作是否依赖数据库模式信息、是否改变统计口径、是否需要复杂查询结构支持，以及其直接作用对象是数据库还是结果集。满足前述任一查询层特征的操作应下沉到查询层，否则保留在分析层。表4-5给出了相应的层级判定矩阵。

表 4-5 任务分解操作的层级判定矩阵

判定维度	查询层特征	分析层特征	判定依据
模式依赖性	需要字段、表关系或类型信息	不再访问底层模式	是否必须依赖 S 完成语义绑定
统计口径依赖性	改变指标定义或聚合口径	不改变既定统计口径	是否需要 L 或预定义业务规则支持
查询结构复杂性	需要关联、子查询或复杂聚合	仅进行轻量结果变换	是否必须通过 SQL 结构求解
操作对象	直接作用于数据库取数过程	直接作用于结果集 $\mathbf{X}$	操作输入是数据库还是查询返回结果

边界情形主要出现在粒度不具体的任务中。例如，“按季度汇总”在已返回月度数据时既可由查询层通过调整分组实现，也可由分析层进一步聚合。针对这类情形，本文采用优先查询层的策略，以减少分析层对原始统计口径的二次加工，并保持结果的一致性与可验证性。

4.4.3 查询结果表示与层间衔接

在完成任务划分后，查询层需要向分析层返回统一的结果表示。本文将 G_q 视为“受约束查询意图进入完整 Text-to-SQL 管线并执行”的上层封装，并将其输出

记为：

$$\mathcal{D} = G_q(x, Q_A; S, K, F) = \langle \mathbf{X}, \Gamma, \sigma \rangle \quad (4-9)$$

其中， $\mathbf{X} \in \mathbb{R}^{n \times m}$ 表示以结构化表格形式封装的查询结果， n 为记录数， m 为字段数； $\Gamma = \{(\gamma_j^{\text{name}}, \gamma_j^{\text{type}}, \gamma_j^{\text{unit}}, \gamma_j^{\text{grain}}) \mid j = 1, \dots, m\}$ 表示各字段的名称、数据类型、单位和粒度等元数据； $\sigma \in \{\text{success}, \text{failure}\}$ 表示查询执行状态。

这一结果表示承担三项作用： $\mathbf{X}$ 为分析算子提供直接操作对象， Γ 为可视化映射和展示规则提供字段语义信息， σ 为流程控制提供执行依据。当 $\sigma = \text{failure}$ 时，分析流程应终止并在结果包的 `logs` 中记录失败原因；当查询执行成功但结果中存在空值、异常粒度或字段说明不足时，系统不再额外设置 `partial` 状态，而是将相关信息写入 Γ 与结果包的 `logs` 中，以保持层间接口简洁稳定。

通过 Q_A 与 $\mathcal{D}$ 两个接口，第三章与第四章形成清晰分工：前者负责可靠取数，后者负责规范分析。图4-4进一步展示了操作判定与层间衔接的整体流程。

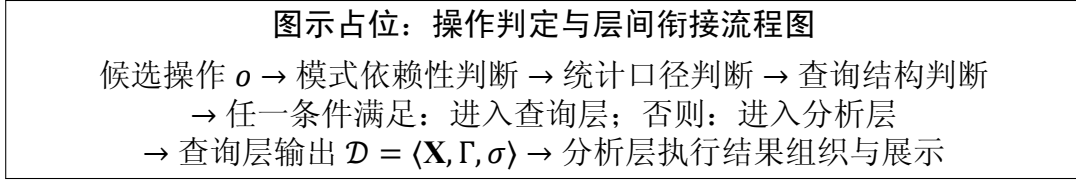


图 4-4 操作判定与层间衔接流程图

4.5 基于分析算子的结果生成方法

在查询层返回结果表示 $\mathcal{D}$ 后，系统需要将结构化数据进一步转化为图表、表格和文字洞察。本节首先说明分析算子的设计与编排机制，随后给出结果生成流程与输出组织方式。

4.5.1 分析算子设计与编排

为保持分析层的可控性，本文引入面向结果组织的轻量分析算子库

$$\mathcal{O}_a = \{\text{sort}, \text{topk}, \text{share}, \text{pivot}, \text{annotate}, \text{format}\} \quad (4-10)$$

其中，`sort` 用于排序，`topk` 用于提取头部或尾部结果，`share` 用于补充占比计算，`pivot` 用于重组对比结果，`annotate` 用于标记关键点，`format` 用于统一数值格式、单位与标签表达。算子仅作用于查询结果的组织与展示，不重新定义业务指标。

为保持查询层与分析层的职责边界，`share` 仅在查询层已返回构成项汇总值的前提下计算占比，不重新定义分子和分母；`annotate` 仅从结果表中抽取峰值、最

低值、拐点和 Top-k 对象等显式特征；`format` 仅对数值单位、百分比形式和标签文本进行规范化处理。

针对四类典型分析任务，本文设计如下基础算子链：

$$\Psi(T) = \begin{cases} [\text{sort}, \text{annotate}], & T = \text{trend} \\ [\text{pivot}, \text{sort}, \text{annotate}], & T = \text{comparison} \\ [\text{sort}, \text{topk}, \text{format}], & T = \text{ranking} \\ [\text{share}, \text{sort}, \text{annotate}], & T = \text{structural} \end{cases} \quad (4-11)$$

其中，趋势分析强调时间顺序与关键点标注，对比分析强调结果对齐与差异突出，排名分析强调排序和头尾识别，结构分析则在组成部分已完整返回的前提下补充占比与重点类别标注。

设调整后的算子链记为 $\text{chain} = [o_1, o_2, \dots, o_l]$ ，则分析层对查询结果的处理过程可统一表示为：

$$\mathbf{Y} = o_l \circ o_{l-1} \circ \dots \circ o_1(\mathbf{X}, \Gamma) \quad (4-12)$$

其中 $\mathbf{Y}$ 表示算子链执行后的结果表。

在图表选择上，本文采用“初始规格 + 数据后校正”的策略。规划阶段给出初始可视化规格 $\hat{V}$ ，结果返回后再结合类别数、字段类型和时间粒度确定最终图表类型：

$$v^* = \begin{cases} \text{line_chart}, & T = \text{trend} \\ \text{bar_chart}, & T \in \{\text{comparison}, \text{ranking}\} \\ \text{pie_chart}, & T = \text{structural} \text{ 且类别数} \leq 6 \\ \text{stacked_bar_chart}, & T = \text{structural} \text{ 且类别数} > 6 \text{ 或存在时间对比} \end{cases} \quad (4-13)$$

在具体编排时，系统以上游规划给出的分析类型 T 、分析需求投影 $P_A = \langle O_a, \hat{V}, \hat{S} \rangle$ 以及查询结果表示 $\mathcal{D} = \langle \mathbf{X}, \Gamma, \sigma \rangle$ 为输入，先由 $\Psi(T)$ 确定基础算子链，再结合 O_a 中的显式要求以及 Γ 中的字段类型、类别数和粒度信息，对算子顺序和图表规格进行轻量调整，最终得到可执行的算子序列与可视化配置。由此，分析层既保留了类型驱动的稳定性的，又能够对真实数据特征作出必要响应。

4.5.2 结果生成流程与输出组织

在完成算子编排与可视化决策后，系统进入结果生成阶段。该阶段以分析需求投影 $P_A = \langle O_a, \hat{V}, \hat{S} \rangle$ 和查询结果表示 $\mathcal{D} = \langle \mathbf{X}, \Gamma, \sigma \rangle$ 为输入，经过执行状态检查、

算子链执行、图表规格修正、摘要生成与结果封装五个步骤，最终输出标准化结果包 R 。算法4.2给出了该流程的详细步骤。

为减少摘要生成过程中的自由发挥，本文将文字洞察生成拆解为“结构化事实抽取”和“摘要组织”两个顺序阶段：

$$F^* = \text{ExtractFacts}(\mathbf{Y}, \hat{S}), \quad I = \text{Verbalize}(F^*, \hat{S}) \quad (4-14)$$

其中， F^* 表示从结果表中抽取的峰值、增幅、排名和占比等结构化事实集合， I 表示在摘要要求约束下形成的文字洞察。

算法 4.2: 分析结果生成算法

Input: 分析需求投影 $P_A = \langle O_a, \hat{V}, \hat{S} \rangle$ ，查询结果表示 $\mathcal{D} = \langle \mathbf{X}, \Gamma, \sigma \rangle$ ，分析类型 T ，可视化规则库 $\mathcal{V}$

Output: 结果包 $R = \{\text{charts}, \text{tables}, \text{insights}, \text{logs}\}$

```

1 if  $\sigma = \text{failure}$  then
2    $R \leftarrow \{\text{charts} = \emptyset, \text{tables} = \emptyset, \text{insights} = \emptyset, \text{logs}(\sigma, \text{错误原因})\};$ 
3   return  $R$ ; // 查询失败，直接终止
4 end
   // 步骤 1: 选择并执行算子链
5  $\text{chain} \leftarrow \Psi(T);$  // 由分析类型确定基础算子链
6  $\text{chain} \leftarrow \text{Adjust}(\text{chain}, O_a, \Gamma);$  // 结合显式要求与字段元数据调整
7  $\mathbf{Y} \leftarrow \text{Apply}(\text{chain}, \mathbf{X}, \Gamma);$  // 依次执行算子，得到处理后结果
   // 步骤 2: 修正可视化规格
8  $n_{\text{cat}} \leftarrow \text{CountCategories}(\mathbf{Y}, \Gamma);$  // 统计实际类别数
9  $\hat{V}' \leftarrow \text{RefineVis}(\hat{V}, \mathbf{Y}, \Gamma, n_{\text{cat}}, \mathcal{V});$  // 按公式(4-13)校正图表类型
   // 步骤 3: 抽取结构化事实并生成文字洞察
10  $F^* \leftarrow \text{ExtractFacts}(\mathbf{Y}, \hat{S});$  // 抽取峰值、增幅、Top-k、占比等事实
11  $I \leftarrow \text{Verbalize}(F^*, \hat{S});$  // 按摘要要求组织文字洞察
   // 步骤 4: 封装结果包
12  $R \leftarrow \{\text{charts}(\hat{V}', \mathbf{Y}), \text{tables}(\mathbf{Y}), \text{insights}(I), \text{logs}(\sigma, \Gamma)\};$ 
13 return  $R$ 
```

算法4.2有四个关键设计要点。第一，执行状态 σ 的前置检查确保了分析层不会在错误数据上盲目执行算子：当查询失败时直接终止并保留日志，从而避免在无效数据上继续分析。第二，算子链的执行过程采用公式(4-12)所示的确定性组合方式，使系统既遵循类型驱动的基础编排 $\Psi(T)$ ，又能根据 AP-Schema 中用户显式指定的 O_a 和真实数据的字段特征进行微调。第三，文字洞察生成采用公式(4-14)所示的“事实抽取优先”策略，先从结果表中提取峰值、增幅、Top-k 和占比等结构化事实，再在摘要要求 $\hat{S}$ 的约束下组织文本，以降低自由生成导致的结论漂移。第

四，可视化规格修正依据公式(4-13)，在真实类别数 n_{cat} 等数据特征的基础上对初始图表类型进行后校正，避免仅凭自然语言需求过早决定图表类型所带来的偏差。

经过上述流程，系统最终输出的结果包 R 满足公式(4-2)所定义的统一结构，为第五章的系统实现提供了标准化的方法输出接口。

4.6 实验与结果分析

本节旨在验证本章方法的三项核心主张：其一，基于 AP-Schema 的分析规划是否能够提升分析任务完成率与结果稳定性；其二，复用查询生成管线是否能够显著提升数据获取正确性并最终改善分析结果质量；其三，方法中各子模块是否各自具有不可替代的独立贡献。与面向 Text-to-SQL 的大规模基准评测不同，本章实验以中等规模任务集上的端到端验证为主，并通过分类型对比、细粒度模块消融和规划质量评估等多维度实验，体现方法的协同效果与各模块的独立作用。

4.6.1 实验设置与评价指标

实验任务基于第三章构建的同源政务数据库场景展开（见表3-3），仍使用财政预算、公共资源和政务项目等领域的 5 个数据库、33 张数据表作为底层数据来源。在此基础上，本文构建了 56 个自然语言分析任务，其中趋势分析 16 个、对比分析 14 个、排名分析 14 个、结构分析 12 个。每个任务均配备人工核验的参考分析目标，包括目标指标、期望图表类型和关键结论要点。

为保持与第三章的衔接，本章在分析规划阶段采用与第三章相同的 Qwen3-Coder-30B-A3B-Instruct 作为基础模型；数据获取环节直接复用第三章的完整查询生成管线。文字洞察组织阶段仍采用同一基础模型，但其输入被约束为公式(4-14)抽取的结构化事实集合。实验评价采用以下指标。

端到端评价采用四个指标：

流程成功率（Pipeline Success Rate, PSR），表示系统能够完成从分析规划、查询获取到结果生成整个流程并返回有效结果包的比例：

$$\text{PSR} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{pipe}_i = 1] \quad (4-15)$$

分析任务完成率（Task Completion Rate, TCR），表示不仅流程执行成功，而且输出图表、表格和摘要在逻辑上满足原始分析意图的比例：

$$\text{TCR} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{task}_i = 1] \quad (4-16)$$

结果正确率 (Result Accuracy, RA), 表示在任务完成样本中, 系统输出与人工参考结果一致的比例。对于数值结果, 若相对误差不超过 $\epsilon = 5\%$ 则视为正确:

$$RA = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbb{I}[\delta(y_i^{pred}, y_i^{ref}) \leq \epsilon] \quad (4-17)$$

其中 N_c 表示任务完成的样本数, $\delta(\cdot)$ 表示相对误差或人工一致性判别函数。

图表恰当性 (Chart Appropriateness, CA), 由 3 名标注者从图表类型选择、坐标映射、标题与标签完整性三个维度进行 1 到 5 分评分, 取平均值:

$$CA = \frac{1}{3N_c} \sum_{i=1}^{N_c} \sum_{j=1}^3 \text{score}_{ij} \quad (4-18)$$

为单独评价 AP-Schema 构建阶段的规划质量, 本文引入以下三个辅助指标。

意图识别准确率 (Intent Accuracy, IA), 表示正确识别分析类型 T 的比例:

$$IA = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[T_i^{pred} = T_i^{ref}] \quad (4-19)$$

槽位填充完备率 (Slot Completeness, SC), 表示 AP-Schema 中各必填字段被正确填充的比例。对于每个任务 i , 设 AP-Schema 共有 $|\mathcal{S}|$ 个必填槽位, 其中正确填充的槽位数为 s_i :

$$SC = \frac{1}{N} \sum_{i=1}^N \frac{s_i}{|\mathcal{S}|} \quad (4-20)$$

约束满足率 (Constraint Satisfaction Rate, CSR), 表示生成的 AP-Schema 通过公式(4-5)完备性校验的比例:

$$CSR = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{Valid}(A_i) = \text{true}] \quad (4-21)$$

为验证本章提出的两项核心设计, 本文设置如下三组方法进行比较:

(1) **Direct-Analysis**: 直接将自然语言分析需求端到端映射为分析过程与结果, 不经过显式的 AP-Schema 规划, 也不显式复用第三章查询生成管线;

(2) **Plan+Free-SQL**: 采用 AP-Schema 和结果生成规范, 但在数据获取阶段不调用第三章查询生成管线, 而是在分析流程中自由生成 SQL;

(3) **Ours**: 本文完整方法, 即 “AP-Schema 规划 + 查询增强取数 + 轻量分析算子结果生成”。

4.6.2 结果对比

图4-5给出了三种方法在 56 个政务分析任务上的总体结果。以 56 个任务为分母，Direct-Analysis、Plan+Free-SQL 和本文方法的流程成功任务数分别为 32、42 和 52，任务完成数分别为 22、32 和 48；在完成的任务样本中，三种方法得到正确分析结果的样本数分别为 16、26 和 46。可以看出，本文方法在流程成功率、任务完成率、结果正确率和图表恰当性四个指标上均明显优于两个对比方法。

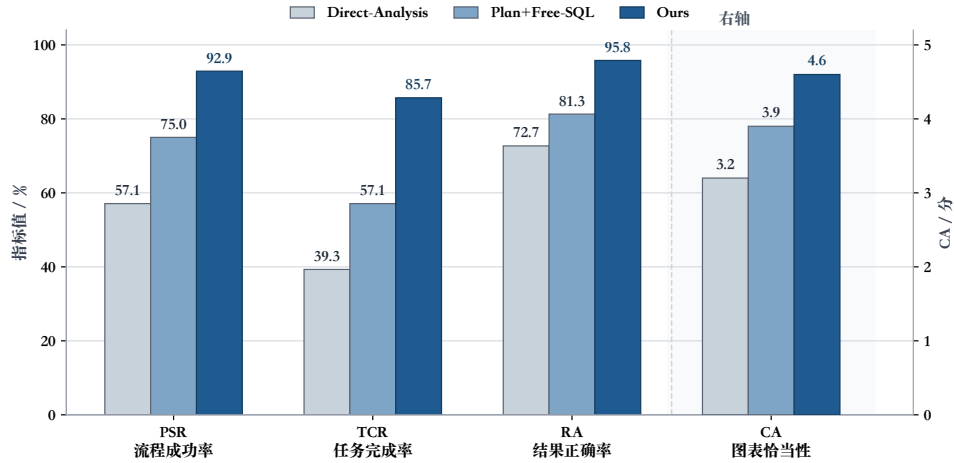


图 4-5 不同分析方法的总体实验结果

分析规划显著提升了任务完成稳定性。与 Direct-Analysis 相比，Plan+Free-SQL 在 PSR 上提升了 17.9 个百分点，在 TCR 上提升了 17.8 个百分点。这表明 AP-Schema 能够有效约束系统对指标、维度、时间粒度和展示形式的理解，减少流程中遗漏关键分析要求的情况。

复用查询生成管线是提升整体性能的关键。在已经采用分析规划的前提下，Ours 相较于 Plan+Free-SQL 在 TCR 上进一步提升 28.6 个百分点，在 RA 上提升 14.5 个百分点。这说明若在分析阶段重新自由生成 SQL，仍会重新暴露字段误解、连接错误和过滤条件偏差等问题；而查询生成管线能够提供更可靠的数据输入，使本章方法将注意力集中在分析本身。

图表恰当性受益于轻量算子与规则约束。Ours 在 CA 上达到 4.6 分，说明通过任务类型到图表类型的规则映射，以及结合数据特征的后校正机制，系统生成的图表在类型选择、标题描述和坐标配置上更加规范。

4.6.3 分类型性能对比

为进一步揭示不同分析场景下各方法的表现差异，本文按四种分析类型分别统计三种方法的 TCR 和 RA。表4-6给出了分类型的实验结果。

表 4-6 不同分析类型下各方法的 TCR 与 RA 对比

方法	趋势分析 (16)		对比分析 (14)		排名分析 (14)		结构分析 (12)	
	TCR	RA	TCR	RA	TCR	RA	TCR	RA
Direct-Analysis	31.3	60.0	42.9	83.3	50.0	71.4	33.3	75.0
Plan+Free-SQL	50.0	75.0	64.3	88.9	64.3	77.8	50.0	83.3
Ours	87.5	92.9	85.7	100.0	92.9	92.3	75.0	100.0

趋势分析对分析规划的依赖最为明显。趋势分析要求系统正确识别时间维度、统计粒度（月度/季度）和同比口径等多重约束。Direct-Analysis 在此类型上的 TCR 仅为 31.3%，说明在缺乏显式规划时，系统经常遗漏时间粒度或同比要求。引入 AP-Schema 后（Plan+Free-SQL），TCR 提升至 50.0%；而本文完整方法进一步将 TCR 提升至 87.5%，表明查询增强机制能够有效保障逻辑指标的正确计算。

对比分析与结构分析受益于查询增强取数最为显著。在这两类任务中，本文方法的 RA 均达到 100.0%，而 Plan+Free-SQL 的 RA 分别为 88.9% 和 83.3%。对比分析要求系统在同一口径下获取多个对象的数据，结构分析需要获取完整的组成项汇总值，二者对查询结果的完整性和一致性有较高要求。自由生成 SQL 时容易出现过滤条件偏差或聚合范围不一致，导致数据缺失或口径错误。

排名分析的 TCR 最高。本文方法在排名分析上的 TCR 达到 92.9%，在四类任务中最高。这主要因为排名分析的结构相对规整（排序 + Top-k），AP-Schema 的约束和算子链 $\Psi(\text{ranking}) = [\text{sort}, \text{topk}, \text{format}]$ 能够直接映射为稳定的执行流程。

结构分析的 TCR 相对偏低，但 RA 最高。结构分析的 TCR 在四类任务中最低，主要原因是部分结构分析任务的分类维度较复杂，规划阶段有时将分类维度误判为对比维度，导致生成的 AP-Schema 类型不匹配。而一旦任务完成其 RA 达到 100.0%，说明在规划正确的前提下，查询增强机制和占比算子能够可靠地产出正确结果。

4.6.4 模块消融实验

总体结果对比已验证了分析规划与查询增强两大核心设计的整体效果。为更精确地量化各子模块的独立贡献，本文在完整方法的基础上分别关闭五个关键子模块，观察端到端指标的变化。与宏观方案对比不同，本节所有消融配置均保留完整的方法框架（AP-Schema + 查询增强 + 结果生成），仅逐一移除特定子模块，以隔离其独立作用。表4-7给出了消融实验结果。

w/o Completeness Validation（去除完备性校验）。保留 AP-Schema 的结构化表示，但跳过公式(4-5)的约束校验与字段补全机制。去除后，TCR 和 PSR 性能下

表 4-7 模块消融实验结果

模型方法	PSR/%	TCR/%	RA/%	CA/分
w/o Completeness Validation	87.5	75.0	92.9	4.4
w/o Task Decomposition	89.3	78.6	86.4	4.1
w/o Operator Chain $\Psi(T)$	91.1	80.4	93.3	3.8
w/o Chart Post-correction	92.9	83.9	95.8	3.9
w/o Fact Extraction	92.9	82.1	93.5	4.0
Full Model	92.9	85.7	95.8	4.6

降。分析发现，失败案例主要集中在趋势分析和结构分析中：前者因缺少时间维度校验导致 G 字段缺失，后者因分类维度未通过约束检查而遗漏关键分组条件。这表明完备性校验是 AP-Schema 从“结构化表示”走向“可靠规划”的关键保障。

w/o Task Decomposition（去除查询/分析层分解）。保留 AP-Schema 和查询增强取数，但不再将操作显式区分为 O_q 和 O_a ，而是将所有操作统一交由大语言模型自行决定分配。去除后，RA 降幅达 9.4 个百分点。错误显示，系统在缺乏显式分层时容易将本应由查询层处理的统计口径操作错误地放入分析层自由计算，导致数值偏差。这说明公式(4-8)所定义的层级判定准则是保证数据正确性的重要机制。

w/o Operator Chain $\Psi(T)$ （去除类型驱动算子链）。保留查询增强取数，但分析层不使用公式(4-11)定义的预设算子链，改由大语言模型自由组织结果展示。去除后，CA 降幅最为明显；TCR 也下降至 80.4%。自由组织模式下，系统在排名分析中经常遗漏 Top-k 截取，在结构分析中未执行占比计算，导致结果不完整。这表明类型驱动的算子链为分析层提供了稳定的执行骨架，避免结构性遗漏。

w/o Chart Post-correction（去除图表后校正）。保留所有其他模块，但图表类型始终使用规划阶段初始化的 $\hat{V}$ ，不再根据实际类别数等数据特征进行校正。去除后，PSR 和 RA 均未受影响，但 CA 从 4.6 分下降至 3.9 分。典型错误包括：当结构分析返回的类别数超过 6 个时仍使用饼图（可读性差），以及当对比维度仅有两个对象时使用多系列柱状图而非更清晰的分组柱状图。这说明公式(4-13)的后校正机制虽不影响数据正确性，但对图表展示的规范性具有显著提升。

w/o Fact Extraction（去除结构化事实抽取）。保留所有其他模块，但文字洞察不经过公式(4-14)的两阶段管线，而由大语言模型直接从结果表生成摘要文本。去除后，TCR 下降至 82.1%，CA 从 4.6 分下降至 4.0 分，RA 也略降至 93.5%。自由摘要模式下，系统偶尔在洞察中引入结果表中不存在的数值（如编造排名位次或增幅百分比），导致标注者判定为任务未完成或结果不正确。这表明“先抽取事实、再组织文本”的约束策略能有效抑制大语言模型在摘要生成中的结论漂移。

4.6.5 AP-Schema 规划质量评估

上述实验从端到端角度评价了整体方法效果，但无法直接反映 AP-Schema 构建过程本身的质量。为此，本文单独评估规划阶段的输出，以验证固定槽位模板与完备性校验机制对规划质量的提升作用。

实验对比两种配置：（1）LLM 直接生成，即不使用固定槽位模板，也不执行约束校验，由大语言模型以自由文本形式输出分析规划；（2）本文方法，即采用固定槽位模板引导草案生成，并通过公式(4-5)进行约束校验与字段补全。评估基于 56 个分析任务，每个任务的参考 AP-Schema 由人工标注确定。表4-8给出了两种配置在意图识别准确率（IA）、槽位填充完备率（SC）和约束满足率（CSR）三个指标上的结果。

表 4-8 AP-Schema 规划质量评估结果

配置	IA/%	SC/%	CSR/%
LLM 直接生成	78.6	67.3	53.6
本文方法	94.6	91.8	91.1

固定槽位模板显著提升了意图识别准确率。LLM 直接生成配置的 IA 为 78.6%，错误主要集中在趋势分析与对比分析的混淆（如将“各区收入变化对比”误判为对比分析而非趋势分析），以及排名分析与结构分析的边界模糊。本文方法通过固定槽位模板强制模型先输出 T 字段再填充后续字段，使 IA 提升至 94.6%。

槽位填充完备率的提升体现了结构化模板的约束作用。LLM 直接生成时，SC 仅为 67.3%，常见遗漏包括分析粒度 G 、查询层操作 O_q 和摘要要求 $\hat{S}$ 等非显式字段。本文方法通过固定槽位顺序和领域知识库辅助填充，将 SC 提升至 91.8%。

约束校验机制是保障规划可执行性的核心。LLM 直接生成的 CSR 仅为 53.6%，即近半数规划不满足表4-3定义的类型约束（如趋势分析缺少时间维度、结构分析缺少分类维度）。本文方法通过完备性校验与二次补全将 CSR 提升至 91.1%，使绝大多数规划在进入查询投影阶段前即满足结构完备性要求。

4.7 本章小结

本章针对政务智能问数中“查到数据后如何规范分析”的问题，提出了基于分析规划与查询增强的政务数据分析方法。该方法引入 AP-Schema 将开放式分析需求转化为结构化规划，通过查询增强的任务分解机制复用第三章的可靠取数能力，并借助轻量分析算子库与可视化规则实现规范化的结果生成。实验表明，本文方法在任务完成率、结果正确率和图表恰当性上均显著优于对比方法；分类型对比、

细粒度模块消融和规划质量评估三组实验进一步验证了各子模块的独立贡献与设计必要性。由此，第三章解决可靠取数，本章解决规范分析，二者共同为第五章的系统实现提供完整的方法基础。

第 5 章 系统设计与实现

5.1 引言

5.2 系统总体架构设计

5.3 业务流程设计

5.3.1 智能查询业务流程

5.3.2 智能分析业务流程

5.3.3 异常回退与结果追溯流程

5.4 系统数据存储

5.4.1 业务数据源接入与元数据抽取

5.4.2 系统数据库设计

5.4.3 结果、日志与缓存管理

5.5 多智能体协同与上下文管理实现

5.5.1 轻量级多智能体角色设计

5.5.2 查询与分析服务编排

5.5.3 上下文维护与状态流转

5.6 系统功能实现与效果展示

5.7 本章小结

结论与展望

致 谢

“诶实扬华，自强不息。”时光荏苒，在交大度过的七载春秋，不仅是我求学生涯中最宝贵的时光，更是我人生观与价值观成型的关键阶段。回首往昔，心中满是感激与敬畏。

首先感谢我的恩师郑宇教授。郑老师不仅在科研上给予我启发式的指引，更在工作态度与人生认知上教会了我深刻的方法论。郑老师治学严谨、精益求精的学者风范，以及对前沿技术的敏锐洞察，令我受益终生。感恩郑老师在我迷茫时的悉心点拨，这份师恩，我将永远铭记于心，并在未来的职业道路上以此自勉。

感谢何华均师兄和麻志鹏师兄。你们严谨认真的科研精神和青云直上的实践态度，潜移默化地影响着我。感谢你们在我遇到瓶颈时提供的无私指导，让我在科研的道路上走得更加踏实。

研途漫漫，感谢在京东科技度过的充实时光。这里承载了我研究生生涯的大半记忆。特别感谢韩博洋和孟垂实两位老师的悉心指导与包容，让我在工业界实战中飞速成长。感谢张晓龙、王璐璐、赵大燕、黎世骄、陈海乐等师兄师姐的并肩作战与温情陪伴，极大地丰富了我的生活色彩。同时，也感谢师弟吴晨宇、虞杰杰、巴聪聪等在工作与科研中的默契协作。

感恩在清华大学智能产业研究院以及字节跳动的宝贵经历。让我深刻体会到了“敢为极致、勇攀高峰”的精神。这些经历磨炼了我的韧性，也拓宽了我看世界的视野。

特别致谢陪伴我走过七载春秋的同门挚友：徐梓航、崔智源、柴江波、王一凡、温海泉。我们的情谊起笔于那个疫前未染尘埃的仲秋，在拔尖班的初见中定格；又在求职毕业、尘埃落定后的盛夏，于奔赴北上深杭的万水千山间，写下这一阶段的终章。科学研究表明，人体细胞每七年就会完成一次整体的更新；何其有幸，在这场漫长的更迭中，是你们陪我见证了从旧我向新我的重塑与蜕变。其深厚羁绊，不仅是学术路上的并肩，更是我求学岁月里最温情、也最坚韧的收获。

最后，感谢那个曾在多地奔波漂泊、却从未放弃梦想的自己。感谢自己在无数个深夜的坚持，在面对未知与挑战时的勇气。愿此去繁花似锦，归来仍是少年。

参考文献

- [1] Liu J, Lin J, Huang S, et al. A survey of text-to-SQL: advances, challenges, and future directions [J]. The VLDB Journal, 2025, 34(1): 1–35.
- [2] Chen J, Xiao S, Zhang P, et al. BGE M3-embedding: multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation [J]. arXiv preprint arXiv:240203216, 2024.
- [3] Malkov Y A, Yashunin D A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2020, 42(4): 824–836.
- [4] Gao D, Wang H, Li Y, et al. Text-to-SQL empowered by large language models: a benchmark evaluation [C]. Proceedings of the VLDB Endowment, 17(5), 2024: 1132–1145.
- [5] Xie X, Zhang G, Wang F, et al. OpenSearch-SQL: enhancing text-to-SQL with dynamic few-shot and consistency alignment [J]. arXiv preprint arXiv:250214913, 2025.
- [6] Pourreza M, Rafiei D. CHASE-SQL: multi-path reasoning and preference optimized candidate selection in text-to-SQL [J]. arXiv preprint arXiv:241001943, 2024.
- [7] Gao Y, Wang Y, Li Y, et al. XiYan-SQL: a multi-generator ensemble framework for text-to-SQL [J]. arXiv preprint arXiv:241108599, 2024.
- [8] TBD. Placeholder reference [J]. TBD, 2024.
- [9] TBD. Placeholder reference [J]. TBD, 2024.
- [10] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models [C]. Advances in neural information processing systems, 35, 2022: 24824–24837.
- [11] Talaei S, Pourreza M, Chang Y C, et al. CMESS: contextual harnessing for efficient SQL synthesis [J]. arXiv preprint arXiv:240516755, 2024.
- [12] Shinn N, Cassano F, Gopinath A, et al. Reflexion: language agents with verbal reinforcement learning [C]. Advances in neural information processing systems, 36, 2024: 8634–8652.
- [13] TBD. Placeholder reference for isft [J]. TBD, 2024.
- [14] Liu J, Li D, Chen B, et al. Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs [J]. Advances in neural information processing systems, 2025, 37: 61588–61617.

- [15] Yu T, Zhang R, Yang K, et al. Spider: a large-scale human-labeled dataset for complex and cross-database semantic parsing and text-to-SQL task [C]. Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, 2018: 3911–3921.
- [16] Bai J, Bai S, Chu Y, et al. Qwen technical report [J]. arXiv preprint arXiv:230916609, 2024.
- [17] DeepSeek-AI. DeepSeek-V3 technical report [J]. arXiv preprint arXiv:241219437, 2024.
- [18] Li B, Mou Y, Li J, et al. Alpha-SQL: zero-shot text-to-SQL via monte carlo tree search [J]. arXiv preprint arXiv:250217600, 2025.
- [19] Li J, Hui B, others. OmniSQL: synthesizing high-quality text-to-SQL data at scale [J]. arXiv preprint arXiv:250302164, 2025.

附录 A

攻读硕士学位期间取得的研究成果

- 一、攻读硕士学位期间发表的论文及科研成果
- 二、参与科研项目/科研获奖等